

ETAS RTA-BSW v12.7.0



RTA-ETHSEC Stack Reference Guide

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-ETHSEC Stack Reference Guide R01 EN – 05.2025

Contents

1	rba_EthAh	5
1.1	Introduction	5
1.2	Supported Features	6
1.3	Extensions	7
1.4	Limitations	7
1.5	Exclusions	7
1.6	Deviations	7
1.7	API Definition	7
1.8	Configuration Advice	8
1.9	Integration Advice	8
1.10	Hardware Requirements	9
2	rba_EthIke2	11
2.1	Introduction	11
2.2	Supported Features	12
2.3	Extensions	13
2.4	Limitations	13
2.5	Exclusions	14
2.6	Deviations	14
2.7	API Definition	14
2.8	Configuration Advice	15
2.9	Integration Advice	15
2.10	Hardware Requirements	16
3	rba_EthIPsec	17
3.1	Introduction	17
3.2	Supported Features	18
3.3	Extensions	19
3.4	Limitations	19
3.5	Exclusions	20
3.6	Deviations	20
3.7	API Definition	20
3.8	Configuration Advice	20
3.9	Integration Advice	21
3.10	Hardware Requirements	22

4	rba_EthTls	23
4.1	Introduction	23
4.2	Supported Features	25
4.3	Limitations	28
4.4	Limitations Specific to TLS 1.3	29
4.5	API Definition	29
4.6	Configuration Advice	31
4.7	Integration Advice	58
4.8	Hardware Requirements	59
5	Glossary	60

1 rba_EthAh

Module name	rba_EthAh
Package	RTA-ETH-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

1.1 Introduction

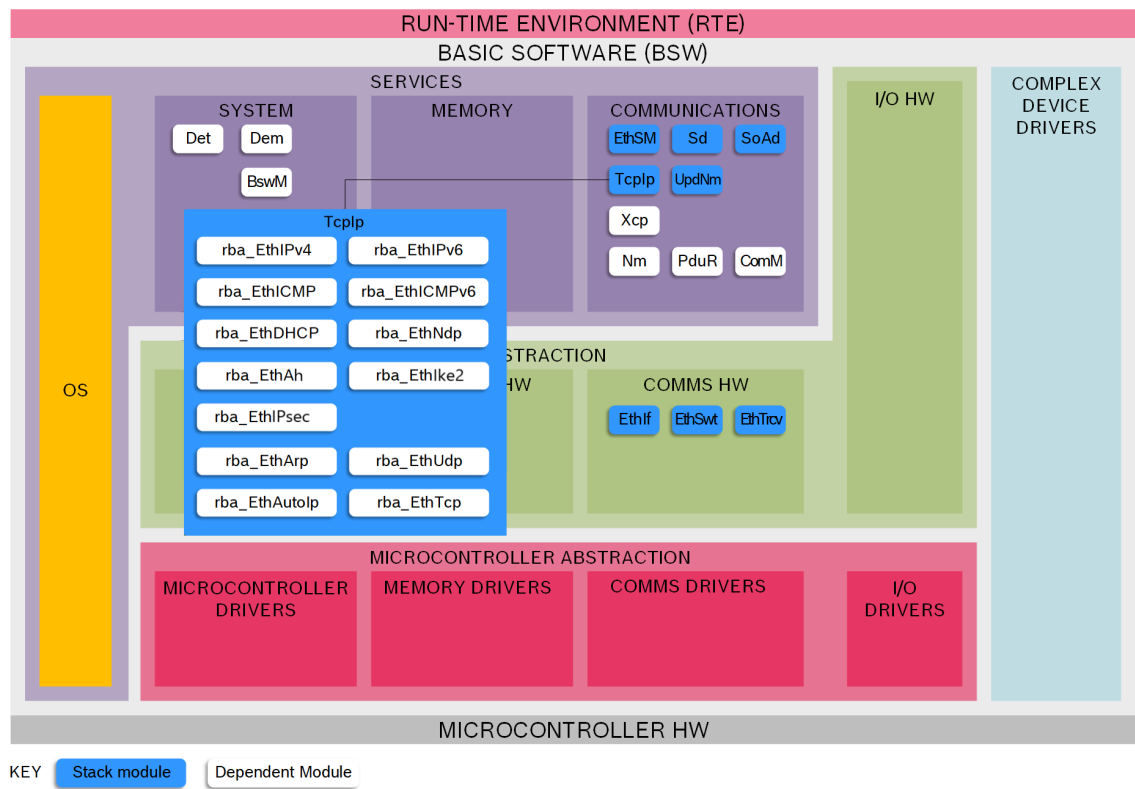


Figure 1.1. RTA-ETH Architecture

The *rba_EthAh* module is located in the communication services abstraction layer, and is considered (in the RTA-ETH-SEC stack) one of the sub-modules of the Tcplp module. Together, they form the 'protocol stack' for TCP/IP messaging and data transfer.

The diagram above shows the position of *rba_EthAh* in the AUTOSAR layered architecture. *rba_EthArp*, *EthIf* and *rba_EthIPv4* are designated as the lower-layer modules with respect to *rba_EthAh*, while *rba_EthIcmp*, *rba_EthUdp* and *rba_EthTcp* are designated as the upper-layer modules.

rba_EthAh's purpose is to implement the Authentication Header (AH) protocol used by the IPsec (Internet Protocol Security) implementation *rba_EthIPsec* (Section 3.1).

It protects communication paths between sender and receiver by offering

connectionless integrity, data origin authentication and protection against replays.

The module does so via the IP packets' header fields, protecting as many of them as possible with a hash value. Some fields, however, (known as mutable fields) might change during transmission and so cannot be protected.



NOTE

The AH protocol does *not* provide encryption.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

1.2 Supported Features

In providing security for a peer-to-peer connection, rba_EthAh offers the following:

- *Connectionless integrity*: ensuring that data is not modified during transmission.
- *Data origin authentication*: ensuring that the identity of the sender is verified.

rba_EthAh implements the above by adding an AH header (that contains a checksum and digital signature) to an IP packet.

- *Prevention of replay attack*: ensuring the rejection of duplicate packets, which can interrupt an IP session. This is done via the use of a *Sequence Number*, which is placed in a header field that is checked by the receiver.
- *Security Associations*: SAs establish the use of a shared security service in the communication path - the 'tunnel' - between sender and receiver. They are, in essence, a collection of algorithms/parameters needed for authentication. SAs are uni-directional in nature, meaning that two are required for two-way communication. They are stored in a Security Association Database (SAD) and each is identified by a Security Parameter Index (SPI).



NOTE

New SAs are created before old ones are deleted.

- *Extended Sequence Numbers*: ESNs are 64-bit Sequence Numbers used for high-speed communication.
- *Re-synchronization*: if there are packet losses in a SA between sender and receiver, subsequent authentication failures are remedied by re-sending a 'bad' packet (the number of re-tries is limited).
- AHs are processed in *Transport Mode* for unicast IPv4 traffic. This means

that the AH header is inserted after the IP Header but before an upper-layer protocol (e.g. TCP, UDP, ICMP) and the data payload.



NOTE

Other implementations of the Authentication Header protocol can use *Tunnel Mode*, in which a *new* IP header is created, with a different sender and receiver address. Tunnel Mode can therefore be used with gateways at either end of a Security Association.

1.3 Extensions

- No known extension parameters are defined in this version.

1.4 Limitations

- Only the RBA_ETHIPSEC_INTEGTFM_HMAC_SHA2_256_128 integrity transform is supported.
- Post-build variant is not supported.

1.5 Exclusions

- There are no known exclusions in this version.

1.6 Deviations

- There are no known deviations in this version.

1.7 API Definition

Table 1.2.rba_EthAh Supported API functions

rba_EthAh_Init	Initializes the rba_EthAh module.
rba_EthAh_ProvideTxBuffer	Provides transmission buffer access to the specified EthIf controller.
rba_EthAh_Transmit	Triggers transmission of a previously filled IP packet buffer obtained from rba_EthAh_ProvideTxBuffer.
rba_EthAh_RxIndication	Gives both an indication and the data of a received IP packet.
rba_EthAh_rba_EthIke2_AllocateIsa	Allocates a new input SA.
rba_EthAh_rba_EthIke2_UpdateIsa	Updates a previously created input SA.
rba_EthAh_rba_EthIke2_ReleaseIsa	Sets free the specified input SA.
rba_EthAh_rba_EthIke2_AllocateOsa	Allocates a new output SA.
rba_EthAh_rba_EthIke2_UpdateOsa	Updates a previously created output SA.

rba_EthAh_rba_EthIke2_Release0sa	Sets free the specified output SA.
rba_EthAh_rba_EthIke2_GetSeqNum	Returns the current value of the Sequence Number associated with the specified input SA.
rba_EthAh_Cbk_MacGenerateNotification	Callback used by Csm to notify rba_EthAh on the result of a finished MAC generation job.
rba_EthAh_Cbk_MacVerifyNotification	Callback used by Csm to notify rba_EthAh on the result of a finished MAC verification job.
rba_EthAh_MainFunctionTx	Passes all the authenticated IP packets to be transmitted to the lower layer modules.
rba_EthAh_MainFunctionRx	Passes all the authenticated IP packets to be received from the lower layer modules.

1.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

1.9 Integration Advice

1.9.1 General

- rba_EthAh does not have its own EcucParamDef file, and is dependent on both the Tcplp and rba_EthIPsec modules.
- Run the configuration validation script and fix errors/warnings (if any).
- Run Code Generation.

1.9.2 Exclusive Areas

Inline functions required for Exclusive Areas in the rba_EthAh module are provided in the template file rba_EthAh_Cfg_SchM.h_tpl.

The implementation of the Exclusive Areas in the inline functions is only required for projects that don't utilize RteGen. The integrator will need to create a header file from the template, which must then be stored in the appropriate include directory. Note: The header file is required for rba_EthAh compilation.

Currently the rba_EthAh module uses the following Exclusive areas:

- AddQueueEntry
- ClearQueueEntry
- TxSecurityAssociation

- RxSecurityAssociation
- SequenceNumGen
- AntiReplayWindow.

Please refer to the template file for more details about how the exclusive areas can be implemented.

- SchM_Enter_rba_EthAh_{ExclusiveAreaName} : Invokes a function to enter a local exclusive area. In single core context, disable the interrupts/tasks preemption.
- SchM_Exit_rba_EthAh_{ExclusiveAreaName} : Invokes a function to exit a local exclusive area. In single core context, enabled the interrupts/tasks preemption.

1.9.3 Init Functions

rba_EthAh_Init can be called by EcuM/BswM, or the integration code, during initialization.

1.9.4 Delnit Functions

No Delnit is required.

1.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthAh module uses the following OS resources:

- rba_EthAh_AddQueueEntry(uint8idLock_u8)
- rba_EthAh_ClearQueueEntry(uint8idLock_u8)
- rba_EthAh_TxSecurityAssociation(uint8idLock_u8)
- rba_EthAh_RxSecurityAssociation(uint8idLock_u8)
- rba_EthAh_SequenceNumGen(uint8idLock_u8)
- rba_EthAh_AntiReplayWindow(uint8idLock_u8)

1.10 Hardware Requirements

Information relating to the hardware resources required by this module is not

currently available.

2 rba_EthIke2

Module name	rba_EthIke2
Package	RTA-ETH-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

2.1 Introduction

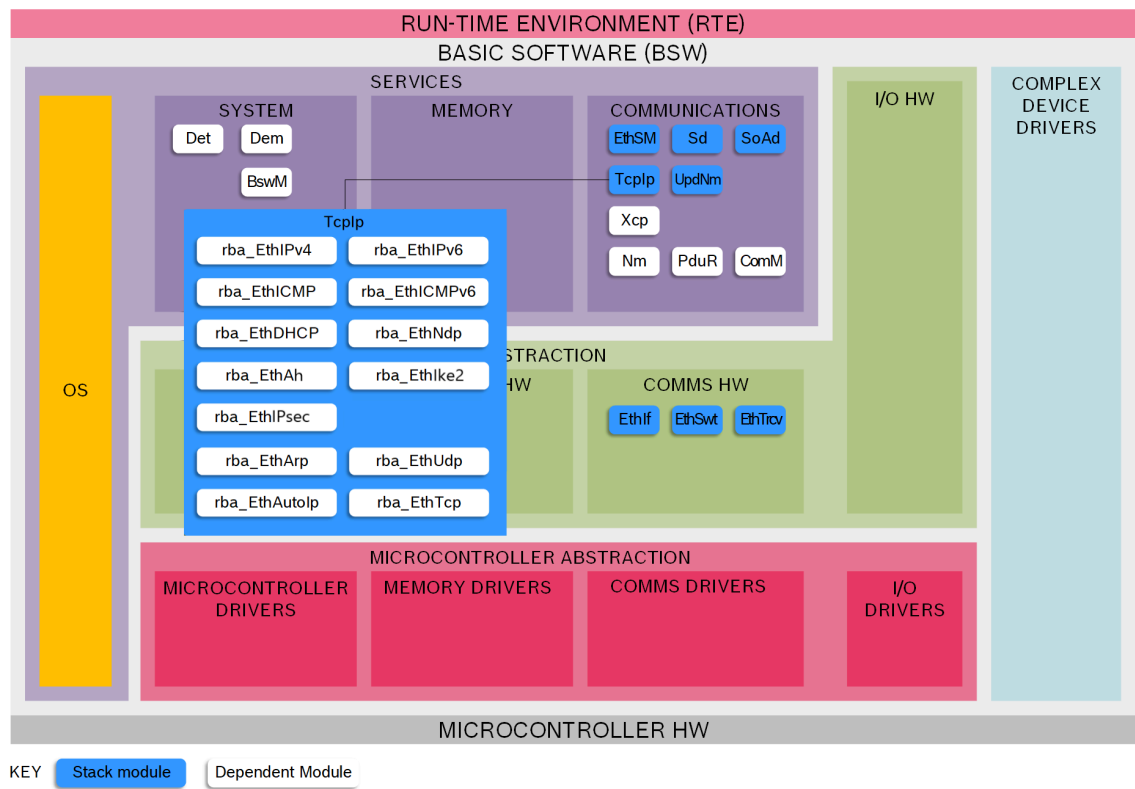


Figure 21.RTA-ETH Architecture

The *rba_EthIke2* module is located in the communication services abstraction layer, and is considered (in the RTA-ETH-SEC stack) one of the sub-modules of the Tcplp stack. Together, they form the 'protocol stack' for TCP/IP messaging and data transfer.

The diagram above shows the position of *rba_EthIke2* in the AUTOSAR layered architecture. *rba_EthUdp* and Tcplp are designated as the lower-layer modules with respect to *rba_EthIke2*.

rba_EthIke2's purpose is to provide the IKE (Internet Key Exchange) protocol as part of an IPsec implementation that protects communication paths between peers i.e. the sender and receiver.

rba_EthIke2 is triggered by *rba_EthIPsec* to find an existing Security Association (see Section 1.2), or start a SA negotiation (Section 2.2.1), when the upper layer

requests the transmission of an IP packet.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

2.2 Supported Features

2.2.1 General

rba_Ethlke2 offers a minimal implementation of the Internet Key Exchange protocol and supports the following:

- Cryptographic algorithm negotiation, pro-active re-authentication, traffic selector negotiation, nonces, address and port agility.
- Extended Sequence Numbers (ESNs).
- Algorithms ENCR_AES_CBC, PRF_HMAC_SHA2_256, AUTH_HMAC_SHA2_256_128 and Curve25519.
- The priority order of the algorithms used is according to the order in which the IKE proposals are configured. The first configured IKE proposal has the highest priority.
- Authentication of IKE Security Association (Section 1.2) using pre-shared keys.
- Child SAs based on AH in Transport Mode.
- A configurable Peer Authentication Database (PAD).
- Deletion of all SAs on shutdown.
- INFORMATIONAL exchange used to delete an SA.
- Generating keying material for an IKE SA and its *Perfect Forward Secrecy*.
- Communication over UDP on port 500.
- Retransmission timers.
- Sequence numbers for message ID.
- A window size of one for overlapping requests.
- State synchronization and connection timeouts.
- Version numbers and forward compatibility.
- IKE SA SPIs and cookies.
- Generation of keying material for child SAs.
- Error handling.
- IKE and generic payload headers.
- Security Association, key exchange, identification, authentication, nonce, notify, delete, traffic selector and encrypted payloads.

2.2.2 IKE Protocol

rba_EthIke2 sets up a Security Association (Section 1.2) between peers in two phases:

Phase one - Creation of ISAKMP (Internet Security Association and Key Management Protocol) tunnel

In this phase, the peers negotiate the following:

- A hashing algorithm to verify peer identity.
- The authentication to be performed.
- The Diffie-Hellman (DH) Group determining the strength of key exchange.
- The lifetime of the ISAKMP tunnel.
- The encryption algorithm.

The DH Group is used to share a key before the peers authenticate each other, using the method negotiated above.

The IKE phase one tunnel is bi-directional and provides a secure channel for phase two.

Phase two - Creation of IKE (IPsec) tunnel

In this phase, the peers negotiate the following:

- Which IPsec security protocol to use.
- Whether to use either Transport Mode or Tunnel Mode.
- Which encryption algorithm to use.
- Which authentication algorithm to use.
- The lifetime of the IKE tunnel.
- DH exchange. This is used in phase two for *Perfect Forward Secrecy*, which obliges peers to run the DH exchange again to generate a new shared key.

Once phase two is complete, a Security Association is created and User data can be protected.

There are two versions of IKE: v1 and v2. rba_EthIke2 implements the latter.

2.3 Extensions

- No known extension parameters are defined in this version.

2.4 Limitations

- Only the RBA_ETHIPSEC_INTEGTFM_HMAC_SHA2_256_128 integrity transform is supported.
- Only the RBA_ETHIKE2_PRANDTFM_HMAC_SHA2_256 pseudo-random transform is supported.

- Only the RBA_ETHIKE2_ENCRYPTFM_AES_CBC encryption transform (with 16-byte block length) is supported.
- Only the RBA_ETHIKE2_DIHELTFM_CURVE25519 Diffie-Hellman Group transform is supported.
- The Optional Exchange type CREATE_CHILD_SA (36) is not supported.
- Authentication via the Shared Key Message Integrity Code is only supported.
- Certificates are not supported.
- Post-build variant is not supported.

2.5 Exclusions

- There are no known exclusions in this version.

2.6 Deviations

- There are no known deviations in this version.

2.7 API Definition

Table 2.2.rba_EthIke2 Supported API functions

rba_EthIke2_Init	Initializes the rba_EthIke2 module.
rba_EthIke2_RxIndication	rba_EthIke2 gets an indication and the data of a received IP packet.
rba_EthIke2_LocalIpAddrAssignmentChg	rba_EthIke2 Local Ip Address assignment change.
rba_EthIke2_RunModeRequest	Performs run mode request.
rba_EthIke2_Cbk_MacGenerateNotification	Callback used by Csm to notify rba_EthIke2 about the result of a finished MAC Generate job.
rba_EthIke2_Cbk_MacVerifyNotification	Callback used by Csm to notify rba_EthIke2 about the result of a finished MAC verification job.
rba_EthIke2_Cbk_EncryptNotification	Callback used by Csm to notify rba_EthIke2 about the result of a finished Encrypt Notification job.
rba_EthIke2_Cbk_DecryptNotification	Callback used by Csm to notify rba_EthIke2 about the result of a finished Decrypt Notification job.
rba_EthIke2_rba_EthIPsec_FindOrStartChildSa	Called by rba_EthIPsec to find or start a child SA.
rba_EthIke2_MainFunction	Cyclic Main function that operates the effects of the IKE2 request modes.

2.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

2.9 Integration Advice

2.9.1 General

- rba_EthIke2 does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. *Tcplp_EcucParamDef.arxml* is therefore required for rba_EthIke2 Code Generation.
- Run Code Generation.

2.9.2 Exclusive Areas

Inline functions required for Exclusive Areas in the rba_EthIke2 module are provided in the template file *rba_EthIke2_Cfg_SchM.h_tpl*.

The implementation of the Exclusive Areas in the inline functions is only required for projects that don't utilize RteGen. The integrator will need to create a header file from the template, which must then be stored in the appropriate include directory. Note: The header file is required for rba_EthIke2 compilation.

Currently the rba_EthIke2 module uses the following Exclusive areas:

- *IkeSession_SaEntry*
- *SpdEntry*
- *AllocateFreePool*
- *SchM_Enter_rba_EthIke2_{ExclusiveAreaName}* : Invokes a function to enter a local exclusive area. In single core context, disable the interrupts/tasks preemption.
- *SchM_Exit_rba_EthIke2_{ExclusiveAreaName}* : Invokes a function to exit a local exclusive area. In single core context, enabled the interrupts/tasks preemption.

2.9.3 Init Functions

rba_EthIke2_Init can be called by *EcuM/BswM*, or the integration code, during initialization.

2.9.4 Delnit Functions

No Delnit is required.

2.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_Ethlke2 module uses the following OS resources:

- rba_Ethlke2_IkeSession_SaEntry(uint8idLock_u8)
- rba_Ethlke2_SpdEntry(uint8idLock_u8)
- rba_Ethlke2_AllocateFreePool

2.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

3 rba_EthIPsec

Module name	rba_EthIPsec
Package	RTA-ETH-SEC
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

3.1 Introduction

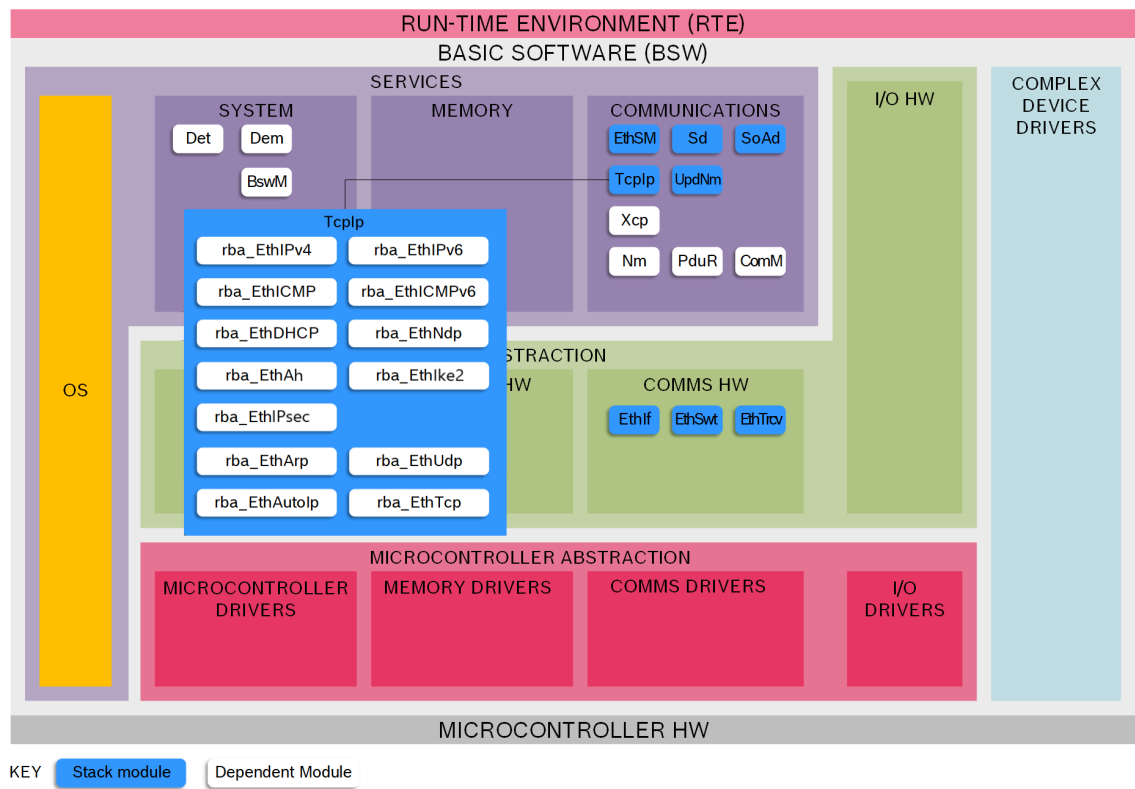


Figure 3.1.RTA-ETH Architecture

The *rba_EthIPsec* module is located in the communication services abstraction layer, and is considered (in the RTA-ETH-SEC stack) one of the sub-modules of the Tcplp stack. Together, they form the 'protocol stack' for TCP/IP messaging and data transfer, with *rba_EthIPsec* offering security.

More specifically, *rba_EthIPsec*'s purpose is to provide an implementation of IPsec (Internet Protocol Security) that protects communication paths between two peers i.e. the sender and receiver.

The diagram shows the position of *rba_EthIPsec* in the AUTOSAR layered architecture. *rba_EthArp*, *EthIf* and *rba_EthIPv4* are designated as the lower-layer modules with respect to *rba_EthIPsec*, while *rba_EthIcmp*, *rba_EthUdp* and *rba_EthTcp* are designated as the upper-layer modules.

Since *rba_EthIPsec* uses the Authentication Header (AH) protocol in Transport

Mode to provide the specific security service, it is used in conjunction with rba_EthAh (Section 1.1).

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

3.2 Supported Features

In protecting IP traffic, rba_EthIPsec provides the following:

3.2.1 Security Policy Database

The Security Policy Database (SPD), established/maintained by a User/System Administrator, defines the requirements for the protection offered by rba_EthIPsec to IP traffic.

The entries in the SPD are matched against the header information (known as *Selectors*) of the IP packets.

The processing actions for these packets are then, in turn, based on the applicable SPD policies identified by the Selectors.

The policies are as follows:

- The IP packet is PROTECTEd using IPsec security services (i.e the Authentication Header protocol).
- The IP packet is DISCARDed.
- The IP packet is allowed to BYPASS IPsec.



NOTE

There is a fourth policy, OPTIONAL, which results in the packet also being DISCARDed.

One SPD is maintained per EthIf controller.

3.2.2 Data Traffic Processing

Outbound Traffic Processing

The following steps are taken:

1. The appropriate SPD is selected, based upon the controller interface.
2. A match is sought between the Selector and an SPD entry (searching in an ascending order of priority).
3. If a matching entry is found, the IP packet will be processed according to the SPD entry's policy, or by the return value of the function referenced by the configuration parameter `TcpIpSecSpdCalloutFunction`:
 - If policy = PROTECT, the key exchange mechanism is invoked to create an outbound Security Association (Section 1.2), or to find one created previ-

ously. Protection will be provided as specified i.e. using the SA and the AH protocol. The IP packet will then be passed to the outbound forwarding function and any data involved in the creation of the SA will be discarded.

- If policy = DISCARD/OPTIONAL, then the IP packet will *not* be transmitted.

4. If there is no match, or policy = BYPASS, then the IP packet will be passed to the outbound forwarding function.

Inbound Traffic Processing

The following steps are taken:

1. The appropriate SPD is selected, based upon the controller interface.
2. If the IP packet is IPsec protected by the AH protocol then:
 - If the IP packet's destination is a unicast address, then the AH protocol maps it to an active SA.
 - If the IP packet's destination is a broadcast address, then an 'Audit Event' is triggered by calling the function referenced by the configuration parameter `TcpIpIpSecAuditEventFunction`.
3. If the IP packet is not IPsec protected, then match is sought between the Selector and an SPD entry (searching in ascending order of priority):
 - If a matching entry is found, then the IP packet will be processed according to the SPD entry's policy, or by the return value of the function referenced by the configuration parameter `TcpIpSecSpdCalloutFunction`. If the policy = PROTECT, then an 'Audit Event' is triggered by calling the function referenced by the configuration parameter `TcpIpIpSecAuditEventFunction`.
 - If there is no match, or the policy = BYPASS, then the IP packet will be passed to the upper layer protocol for further processing.

Please see Configuration Advice (Section 3.8) for more details on the Security Policy Database.

3.3 Extensions

- No known extension parameters are defined in this version.

3.4 Limitations

- Only secured communication over IPv4 is supported. IPv6 support is not yet implemented.
- Only the Authentication Header (AH) protocol is supported. Encapsulating Security Payload (ESP) is not yet implemented.
- Only IPsec end-to-end point Transport Mode is supported. Tunnel Mode is not yet implemented.

- Post build variant is not supported.

3.5 Exclusions

- There are no known exclusions in this version.

3.6 Deviations

- There are no known deviations in this version.

3.7 API Definition

Table 3.2.rba_EthIPsec Supported API functions

rba_EthIPsec_Init	Initializes the rba_EthIPsec module.
rba_EthIPsec_ProvideTxBuffer	Provides access to a transmission buffer for the EthIf controller associated with the specified local address identifier.
rba_EthIPsec_Transmit	Triggers the transmission of a previously filled IP packet buffer obtained from rba_EthIPsec_ProvideTxBuffer.
rba_EthIPsec_IsTransmissionPossible	Provides transmission status of the local IP address, local port, remote IP address and remote port combination.
rba_EthIPsec_RxIndication	Callback giving rba_EthIPsec an indication and the data of a received IP packet.
rba_EthIPsec_rba_EthAhEsp_GetIcvLength	Maps the length of the Integrity Check Value (ICV) to the index of an Integrity Transform.
rba_EthIPsec_rba_EthAhEsp_GetIntegBlockSize	Maps the size of data blocks to the index of an Integrity Transform.
rba_EthIPsec_rba_EthAhEsp_GetConfig	Returns the configuration associated with a specific EthIf controller.
rba_EthIPsec_rba_EthIke2_FindProtectedSpdEntry	Searches the SPD and returns a pointer to the matching SPD entry.

3.8 Configuration Advice

3.8.1 Security Policy Database

- The SPD is specified via the configuration parameter TcpIpSpdEntry.
- The SPD is ordered by the priority maintained in the configuration parameter TcpIpIpSecPriority.
- The security policies i.e. PROTECT, DISCARD, BYPASS, OPTIONAL are set

using the configuration parameter `TcpIpIpSecPolicy`.

After a match, implementation of a security policy uses the following parameters:

- Range of remote (receiver) IP addresses from `TcpIpRemIpAddrStart` to `TcpIpRemIpAddrEnd`.
- Range of local (sender) IP addresses from `TcpIpLocalIpAddrStart` to `TcpIpLocalIpAddrEnd`.
- The next-layer protocol in `TcpIpProtocol`. If the protocol is either TCP or UDP:
 - Range of remote ports from `TcpIpRemPortRangeStart` to `TcpIpRemPortRangeEnd`.
 - Range of local ports from `TcpIpLocalPortRangeStart` to `TcpIpLocalPortRangeEnd`.
- If the protocol is ICMP:
 - Range of remote 16-bit Selector for the ICMP message type and code, from `TcpIpRemPortRangeStart` to `TcpIpRemPortRangeEnd`.
 - Range of local 16-bit Selector for the ICMP message type and code, from `TcpIpLocalPortRangeStart` to `TcpIpLocalPortRangeEnd`.



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

3.9 Integration Advice

3.9.1 General

- `rba_EthIPsec` does not have its own `EcucParamDef` file, and is fully dependent on the `TcpIp` module. *TcpIp_EcucParamDef.arxml* is therefore required for `rba_EthIPsec` Code Generation.
- Run Code Generation.

3.9.2 Init Functions

`rba_EthIPsec_Init` can be called by `Ecum/BswM`, or the integration code, during initialization.

3.9.3 Delnit Functions

No `Delnit` is required.

3.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_EthIPsec module does not use any exclusive OS resources.

3.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

4 rba_EthTls

Module name	Rba_EthTls
Package	RTA-ETHSEC
Module ID	N/A
AUTOSAR Version	22-11

4.1 Introduction

The *Transport Layer Security (TLS)* protocol provides secure communication over the Internet. TLS allows client/server applications an environment that prevents eavesdropping, tampering or message forgery. It provides the secure, confidential and authenticated transmission of an octet stream between programs, running on computers connected to a Local Area Network, Intranet or the public Internet through *TCP/IP*. It resides in the Transport Layer.



NOTE

This module is implemented according to the requirements mentioned in RFC 5246 and RFC 8446.

4.1.1 Interaction

rba_EthTls is an ETAS-specific sub-module of *TCP/IP*, which encapsulates all API calls from the upper layer to *rba_EthTls*. The *rba_EthTls* module transmits and receives TLS fragments to and from *rba_EthTcp*:

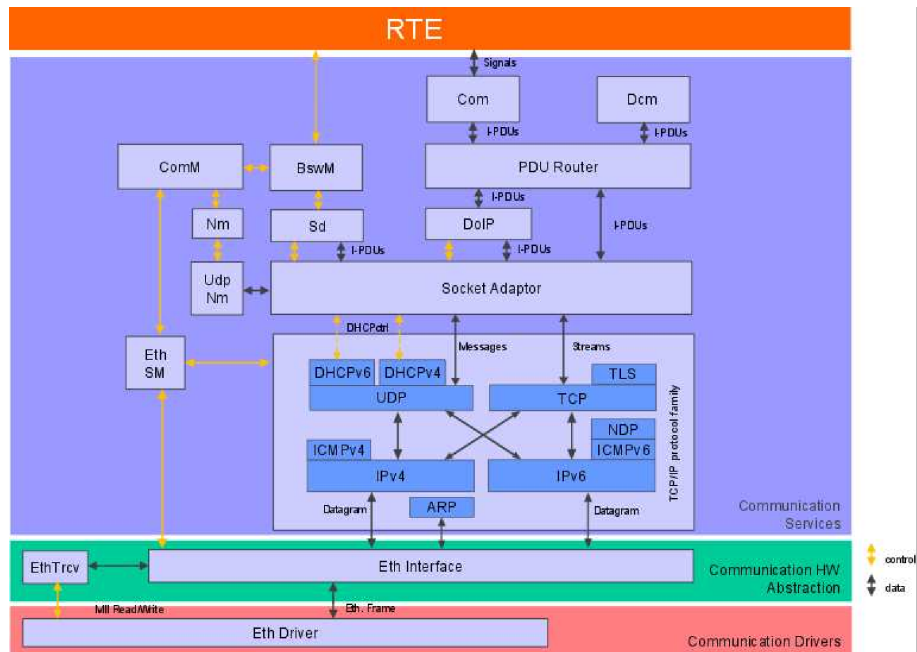


Figure 4.1. Architecture overview

TLS provides communication at an intermediate level of the Transmission Control Protocol (TCP) layer i.e. the Transport Layer. The Socket Adaptor connects the Ethernet layer (or, more precisely, the TCP/IP layer) and the AUTOSAR layer. In the latter, it's usually the case that static-configured PDUs are used to transmit information via the Bus systems. In TCP/IP-based communication, channels are opened and the application directly uses them to transmit data without a specific layout. The TCP/IP concept of data transmission provides dynamic socket-based communication via the Ethernet.

Transport Layer Security accepts data from TCP/IP, encrypts it and adds a *Record Header*. The TLS Fragment is then encapsulated into a TCP/IP segment, and exchanged with peer ECUs. During the TLS handshake, a Fragment will consist of a Record Header, a *Handshake Header* and the Application Data. Respectively, the Record Header and Handshake Header contain five and four mandatory fields. Although the Application Data follows the Handshake Header during the handshake, it follows the Record Header after the handshake has successfully completed.

The basic unit of data in SSL is a record. Each record consists of a five-byte record header, followed by data.

This is the general format of all TLS records.

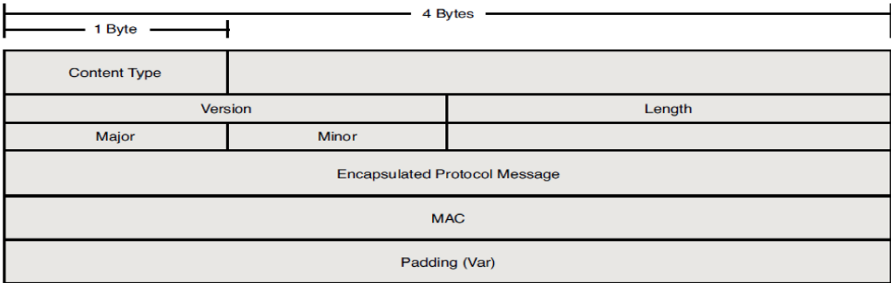


Figure 4.2.TLS Record Header

Handshake Protocol:

Most messages exchanged during the setup of the TLS session are based on this record, unless an error or warning occurs and needs to be signaled by an Alert protocol record or the encryption mode of the session is modified by another record. See the following Handshake Protocol Record:

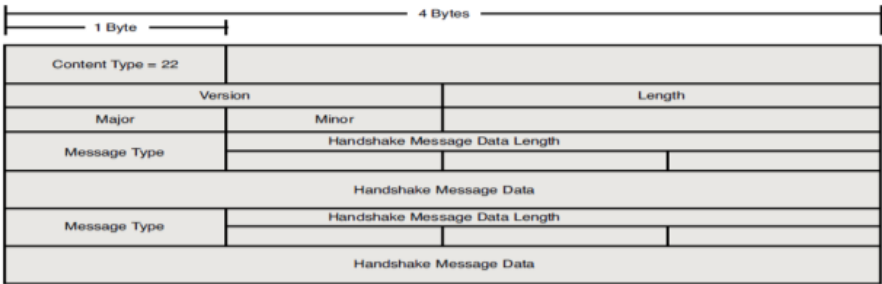


Figure 4.3.TLS Handshake Header

The transmission and reception of data is done via ports, or the *IpAddress*, which are pre-configured for the TLS connection.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

4.2 Supported Features

- TLS protocol framing and parsing (record, alert, change cipher spec., handshake and application data) as per RFC 5246.
- TLS protocol supported version 1.2 as per RFC 5246.
- TLS transmission support (encryption) as per RFC 5246.
- TLS reception support (decryption) as per RFC 5246.
- Rx buffering support to re-assemble the fragmented records and handshake messages.

- Error handling for various alert messages as per RFC 5246.
- Session re-negotiation feature support.
- Session resumption feature supported.
- TLS Server and Client mutual authentication support for all supported ciphersuites.
- Supported ciphersuites:

TLS_PSK_WITH_NULL_SHA256</L-1>

TLS_ECDHE_ECDSA_WITH_NULL_SHA

TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256

TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256

TLS_ECDH_RSA_WITH_AES_128_CBC_SHA256

TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256

TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256

TLS_ECDHE_ECDSA_WITH_AES_128_CCM

TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8

TLS_ECDHE_ECDSA_WITH_AES_256_CCM

TLS_AES_256_GCM_SHA384

- Supported Cipher Algorithms:

AES_128_CBC

AES_256_CBC

AES_128_GCM

AES_256_GCM

CHACHA20_POLY1305

AES_128_CCM

AES_256_CCM

- Supported PRF/Hash Algorithms:

SHA256

SHA384

- Supported MAC Algorithms:

SHA1

SHA256

SHA384

- Supported Signature for Generate and Verify Algorithms:

ecdsa_secp256r1_sha256

ecdsa_secp384r1_sha384

ed25519

RSA

ecdsa_secp521r1_sha512 (Specific to TLS 1.3)

- Supported Key Exchange Curves:

ED25519

SECP384R1

SECP256R1

SECP521R1 (Specific to TLS 1.3)

- TLS complete handshake support with the following extensions:

Max. fragment length extension (512, 1024, 2048, 4096 codes) as per RFC 6066.

Record size limit extension as per RFC 8449.

Sever name indication extension as per RFC 6066.

EC supported groups extension as per RFC 8422.

EC supported point formats extension as per RFC 8422.

Supported signature algorithms extension as per RFC 8422.

Encrypt and then MAC extension as per RFC 7366.

Renegotiation information extension as per RFC 5746.

Session ticket extension as per RFC 5077.

Session Id resumption as per RFC 5246.

Key Share as per RFC 8446.

Supported Versions as per RFC 8446.

Certificate Authorities as per RFC 8446.

- Timeout parameters such as `TcpIpRbTlsWaitForPeerTimeout` (the connection shall be terminated after timeout if the expected handshake messages are received), `TcpIpRbTlsSessionRenegotiationTimeout` (session re-negotiation if the timer expires), `TcpIpRbTlsSessionRenegotiationInitiateTimeout` (the waiting period for a remote response after re-negotiation is initiated), and `TcpIpRbTlsSessionResumptionTimeout` (the session will be renewed if the timer expires) are supported.

- TLS key generation operation can be split into multiple MainFunctions to reduce the runtime.
- Preparation/processing of the TLS finished message operation can be split in multiple MainFunctions to reduce the runtime.
- AEAD ciphers are supported.
- If all CSM jobs are configured with synchronous mode, then `rba_EthTls_BackgroundMainFunction` will be created to perform the TLS functionality since the CSM operation could not be completed within the `rba_EthTls_ConnMainFunction` period.
- TLS async functionality is split into multiple MainFunctions such as `rba_EthTls_MainFunction`, `rba_EthTls_MainFunctionTx` and `rba_EthTls_MainFunctionRx` running in parallel.
- `rba_EthTls_MainFunction`: This scheduled function handles the secure connection establishment and closure (TLS handshake). In other words, it exchanges messages such as *Handshake*, *Change Cipher Spec*, *Alert*.
- `rba_EthTls_MainFunctionTx`: This scheduled function handles the transmission functionality. It is used to transmit the application data alone (handshake messages will not be transmitted from here).
- `rba_EthTls_MainFunctionRx`: This scheduled function handles the reception functionality. It is used to process the received application data alone (handshake messages will not be processed from here).

4.3 Limitations

Version v12.7.0 has following limitations:

- For the TLS Server, the certificate hierarchy has been tested with two levels and three levels for the Server certificate authentication.
- In the Client and Server hello messages, the Random field is filled only with a random number and GMT_Unix_Time is not included.
- Multiple handshakes at any one time are not possible due to HSM constraints (only one instant will be active from the Public Value calculation until the Public Secret calculation).
- TLS connectionGroups are not supported.
- The Record Size Limit extension has not been tested.
- At least a two-level certificates chain should be configured in the KeyM module in order to authenticate remote certificates (server/client).
- If more than one certificate identity is configured, then the one selected for authentication is based upon the user certificate preference (signature algorithm).
- If any CSM job (*CalcPubVal*, *CalcSecret*, *SignatureGenerate*, *SignatureVerify*, *KeyElementSet*) is configured as synchronous, then *CryptoRbHsmTimeout* should be configured such a way that it will not get timed out before completing the requested operation. During functional testing, *Cryp-*

toRbHsmTimeout is configured with 500ms in the *PRP* environment (*Unicon Cut2* platform).

- *Csm_MacGenerate* and *Csm_MacVerify* APIs have been tested for *SHA1* with the *rba_CryptoCCL* library; it is not supported in *rba_CryptoHSM*.
- *Csm_AeadDecrypt* and *Csm_AeadEncrypt* APIs has been tested for *CHACHA20_POLY1305* with *rba_CryptoCCL* library and it is not supported in *rba_CryptoHSM*.
- *Csm_SignatureGenerate* and *Csm_SignatureVerify* APIs has been tested for *RSA* with *rba_CryptoCCL* library and it is not supported in *rba_CryptoHSM*.
- *Csm_AeadDecrypt* and *Csm_AeadEncrypt* APIs has been tested for *AES_CCM* with *rba_CryptoCCL* library and it is not supported in *rba_CryptoHSM*.
- Same TLS Server connecting to multiple TLS remote client in parallel has been tested with *rba_EthTls_BackgroundMainFunction* and it cannot be tested with schedulable main functions (10ms, 20ms and etc) due to HSM limitation where only one instance can run simultaneously.

4.4 Limitations Specific to TLS 1.3

- TLS 1.3 Client Implementation is only supported as per RFC8446.
- Extensions of Received Certificate and Encrypted Extensions will not be processed.
- No extensions will be prepared during Client Certificate message preparation.
- The following limitations are with respect to the *certificate_request_context* field:
 - The processing of this field is not handled by the TLS 1.3 Client.
 - The preparation of this field in the Certificate message is available but not tested.
- The following messages are not currently supported in this implementation of TLS 1.3.
 - *key_update*
 - *end_of_early_data*
- Session Resumption, Session Renegotiation and Post Client Authentication is not supported in TLS 1.3.

4.5 API Definition

List of supported API functions

<i>rba_EthTls_Init</i>	Called from <i>TcpIp</i> to initialize the module.
<i>rba_EthTls_GetConnectionType</i>	Called from <i>rba_EthTcp</i> to return the connection type.

rba_EthTls_GetTlsConnIdxforSecureAddrIPv4	Called from the rba_EthTcp module to determine whether the Ipv4 address is secured and returns the TLS static connection Id.
rba_EthTls_GetTlsConnIdxforSecureAddrIPv6	Called from the rba_EthTcp module to determine whether the Ipv6 address is secured and returns the TLS static connection Id.
rba_EthTls_OpenConnection	Called from the rba_EthTcp module to open the TLS connection if the application requests so.
rba_EthTls_CloseConnection	Called from the rba_EthTcp module to close the TLS connection if the application requests so.
rba_EthTls_ResetConnection	Called from the rba_EthTcp module to reset the TLS connection if the Tcp socket is closed abruptly.
rba_EthTls_ProvidePlainTextBuffer	Called from the rba_EthTcp module to get the plain text buffer.
rba_EthTls_Transmit	Called from the rba_EthTcp module to get encrypted data during transmission of application data.
rba_EthTls_EnableRenegotiation	Called from SoAd via rba_EthTcp to initiate session renegotiation.
rba_EthTls_MainFunction	Performs the rba_EthTls action needed for entry. If RTE is enabled the main function declaration is present in SchM_rba_EthTls.h. If RTE is disabled then the main function prototype should be included in this file since SchM_rba_EthTls.h file won't be generated.
rba_EthTls_MainFunctionTx	This schedule function handles the reception functionality
rba_EthTls_MainFunctionRx	This function shall process the open connection request for the connections which are in established state.
rba_EthTls_BackgroundMainFunction	This schedule function performs the TLS functionality for the connection which couldn't be performed in main
rba_EthTls_TxConfirmation	Callback from the rba_EthTcp module to indicate TLS sent data is acknowledged.
rba_EthTls_RxIndication	Callback from the rba_EthTcp module to indicate reception of TLS messages.

RBA_ETHTLS_CFG_KEYM_ASYNC_CALLBACK	Callback from the KeyM module if the Async operation was triggered and it has been completed.
RBA_ETHTLS_CFG_CSM_ASYNC_CALLBACK	Callback from CSM module if the Async operation was triggered and it has been completed.

4.6 Configuration Advice

This section describes the TLS-relevant parameters of the *TcpIp* configuration. They are present in the *TcpIpConfig.TcpIpTlsConfig* container.



NOTE

The *rba_EthTls* module is fully dependent upon the *rba_EthTcp* and *TcpIp* modules, since *rba_EthTls* does not have its own *EcucParamDef* file. *TcpIp_EcucParamDef.arxml* is required for *rba_EthTls* code generation.

4.6.1 TcpIpTlsConfig

This container specifies the configuration parameters as follows:

TcpIpRbTlsECSupportedGrpsMaxCount: The maximum number of Elliptic curve groups that can be supported if the Elliptic curve-related ciphersuite is selected in the configuration. If this parameter is not configured, then it can be set with a default value of 10.

TcpIpRbTlsECSupportedPointsMaxCount: The maximum number of Elliptic curve point formats that can be supported if the Elliptic curve-related ciphersuite is selected in the configuration. If this parameter is not configured, then it can be set with a default value of 1.

TcpIpRbTlsMaxRtx: The number of times the TLS message can be re-transmitted if the previous transmission was not successful. If this parameter is not configured, then it can be set with a default value of 3.

TcpIpRbTlsMaxRxBufferSizePerConnection: The Rx buffer size configured for each connection. If the TLS payload present in the TCP buffer is fragmented, it is copied to the Rx buffer present in the TLS component. If this parameter is not configured, then it can be set with a default value of 4KB.

TcpIpRbTlsMaxTxBufferSizePerConnection: The Tx buffer size configured for each connection. If the TLS payload present in the TCP buffer is fragmented, it is copied to the Tx buffer present in the TLS component. If this parameter is not configured, then it can be set with a default value of 4KB.

TcpIpRbTlsPresharedKeyIdentityHintMaxLength: The maximum length of the pre-shared key identity hint. It is dependent upon the pre-shared key-related ciphersuites and used only when they are selected. If this parameter is not configured, then it can be set with a default value of 32.

TcpIpRbTlsPresharedKeyIdentityMaxLength: The maximum length of the

pre-shared key identity. It is dependent upon the pre-shared key-related ciphersuites and used only when they are selected. If this parameter is not configured, then it can be set with a default value of 32.

TcpIpRbTlsPresharedKeyMaxLength: The maximum length of the pre-shared key. It is dependent upon the pre-shared key-related ciphersuites and is used only when they are selected. If this parameter is not configured, then it can be set with a default value of 32.

TcpIpRbTlsRTxTimeout: The timeout for re-transmission. It is used only for TLS message re-transmission to the TCP buffer. After the timeout (in seconds), the ECU stops trying to re-transmit and notifies the remote node by sending an Alert message with description code 0x50 (Internal Error).

TcpIpRbTlsServerNameIdentificationMaxLength: The maximum length of the TcpIpTlsServerNameIdentification. If this parameter is not configured, then it can be set with a default value of 64.

TcpIpRbTlsWaitForPeerTimeout: Parameter used to determine the timeout (in seconds) for the TLS message reception. After any timeout, the ECU will stop waiting for the reception of TLS messages and it will notify the remote node with an Alert message of description code 0x50 (Internal Error).

TcpIpTlsCsmRandomGenerateJobRef: Parameter specifying the job referenced to the CSM that will generate the random number required during the handshake for the calculation of the Pre-Master Secret by the client/server.

TcpIpTlsMaxConnections: Defines the maximum number of TLS connections that can be opened at the same time.

4.6.2 RBA_ETHTLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 Ciphersuite Configurations

The configurations, job and key references to be done in the *rba_EthTls* component to *Csm* and *KeyM* are specified as follows:

Configuration : Tls config container

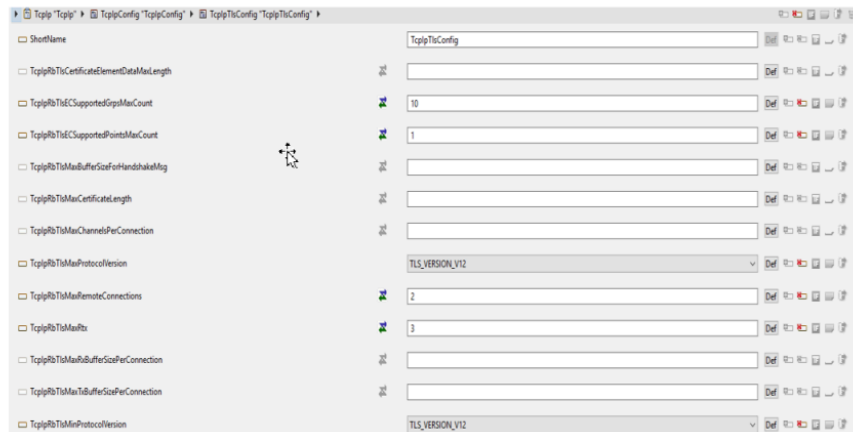


Figure 4.4.rba_EthTls Configurations to be done

Configuration : Tls config container

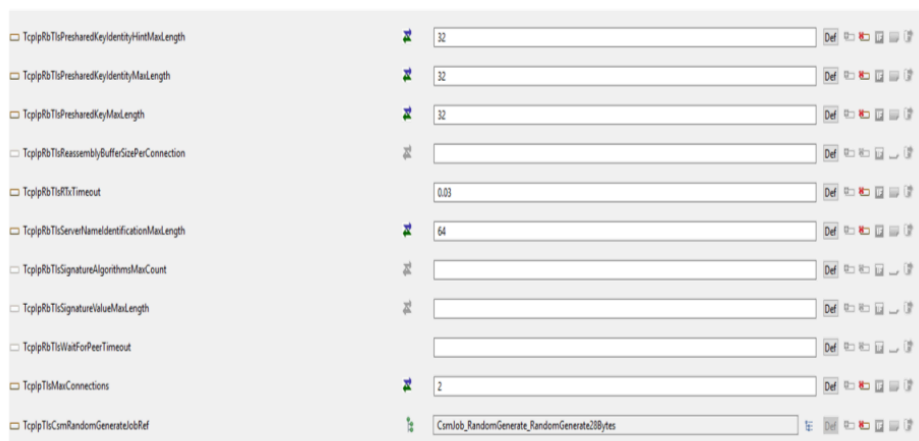


Figure 4.5.rba_EthTls Configurations to be done

Configuration :TcplpTlsCertificateIdentity container

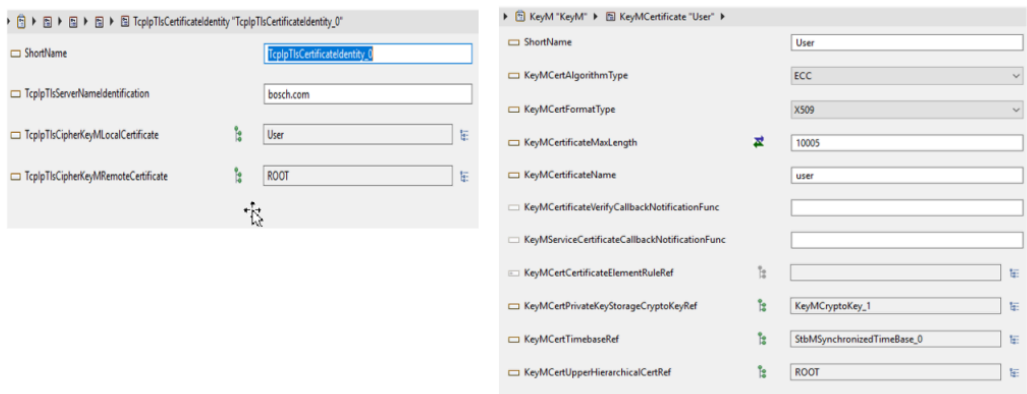


Figure 4.6.rba_EthTls Certificate Configurations to be done

Configuration :TcplpTlsCiphersuiteDefinition container

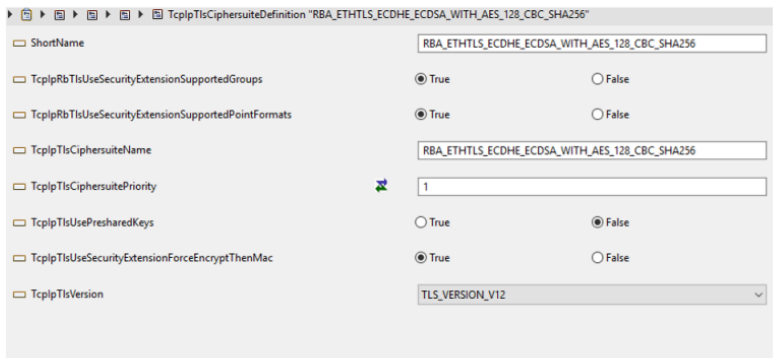


Figure 4.7.rba_EthTls Ciphersuite Definition Configurations to be done

Configuration :TcplpTlsCiphersuiteWorker container

I

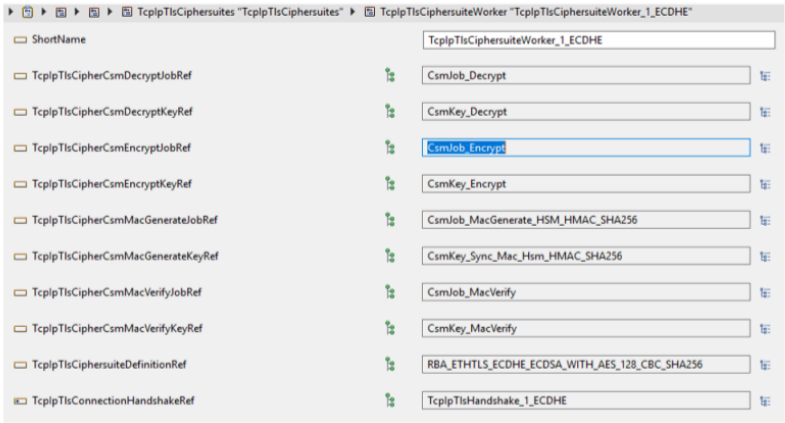


Figure 4.8.rba_EthTls Ciphersuite Worker Configurations to be done

Configuration :TcplpTlsCiphersuiteHandshake container

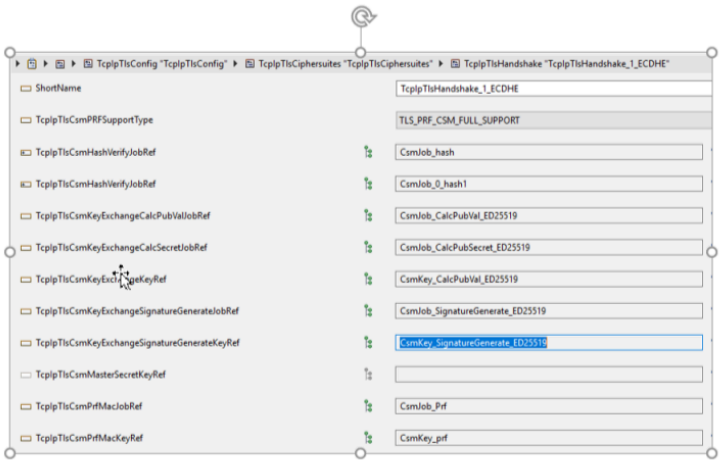


Figure 4.9.rba_EthTls Handshake Container Configurations to be done

Configuration :Csm Public Value Configuration

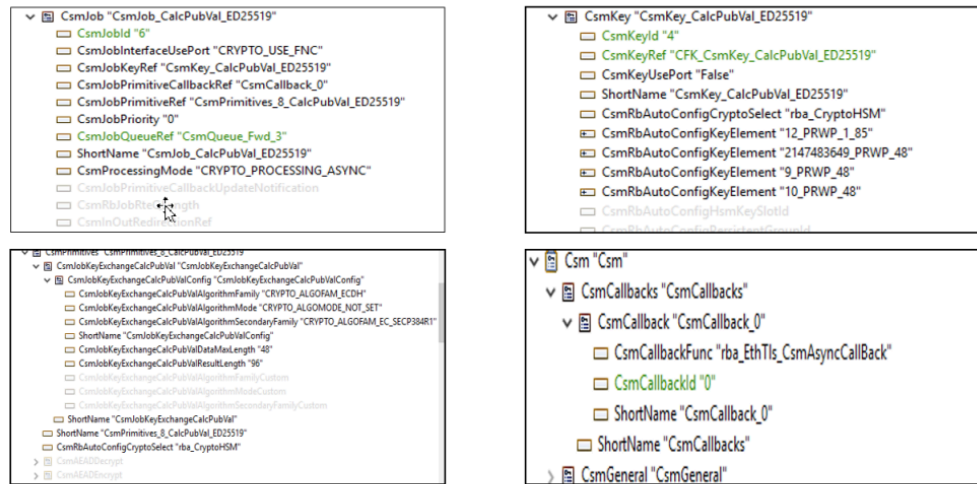


Figure 4.10.Required Configurations for Public Value calculation

Configuration :Csm Public Secret Configuration

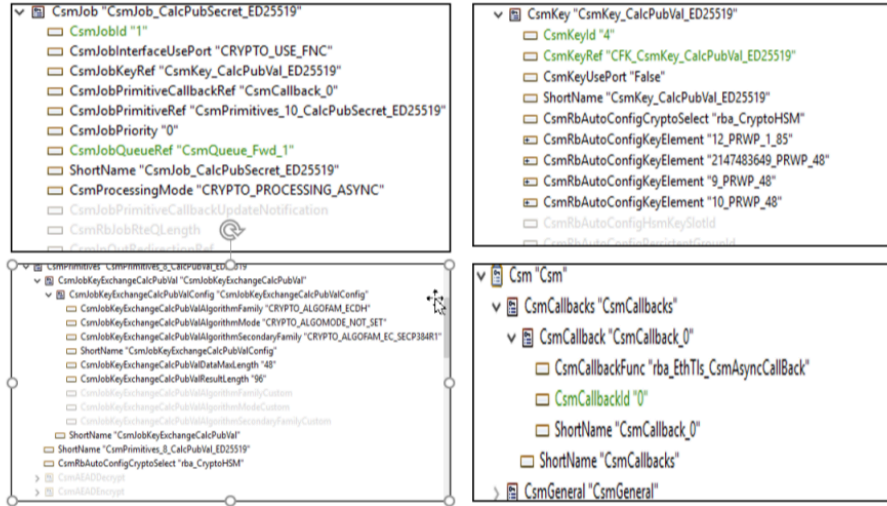


Figure 4.11.Required Configurations for Public Secret calculation

Configuration :Csm Hash Configuration



Figure 4.12.Required Configurations for hash calculation

Configuration :Csm Decrypt Configuration

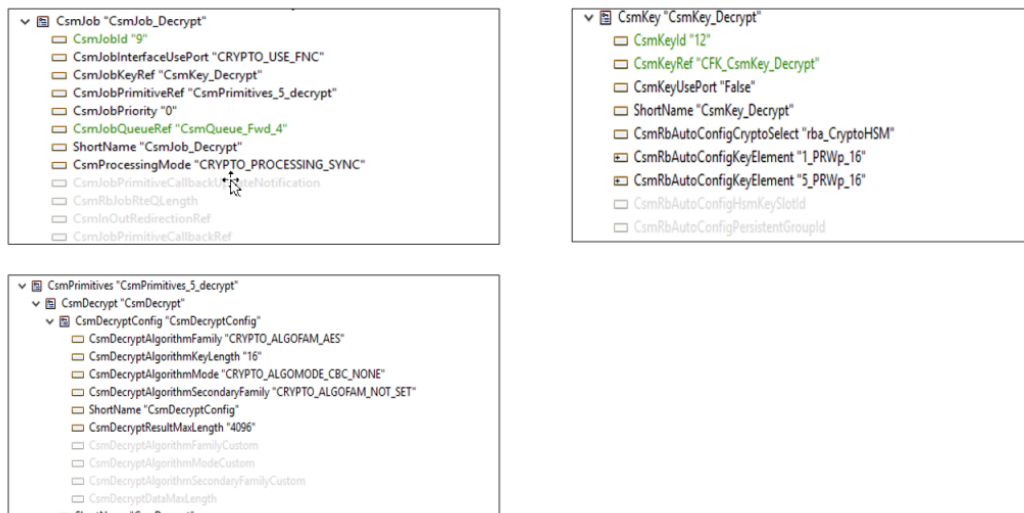


Figure 4.13.Required Configurations for Decryption Process

Configuration :Csm Encrypt Configuration

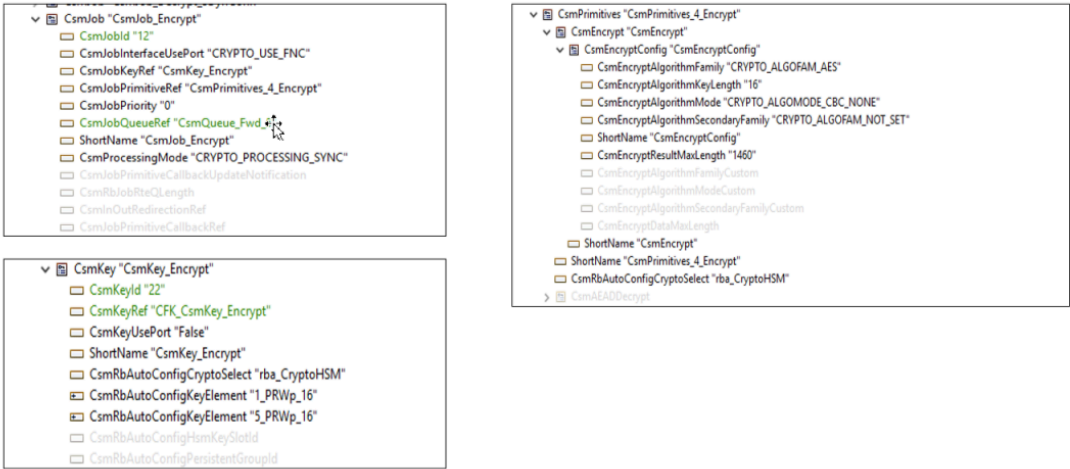


Figure 4.14.Required Configurations for Encryption Process

Error opening image file: [Errno 2] No such file or directory: '/c/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Mac_Generate_Configuration.png'

Figure 4.15.Required Configurations for Generation of MAC

Error opening image file: [Errno 2] No such file or directory: '/c/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Mac_Verify_Configuration.png'

Figure 4.16.Required Configurations for Verification of MAC

Configuration :Csm PRF Configuration

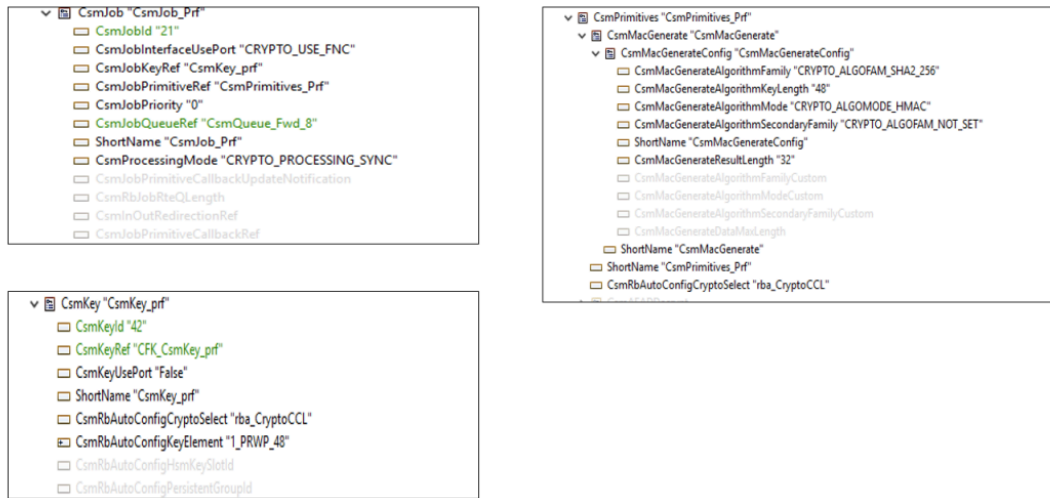


Figure 4.17.Required Configurations for PRF

Configuration :Csm Random Generate Configuration

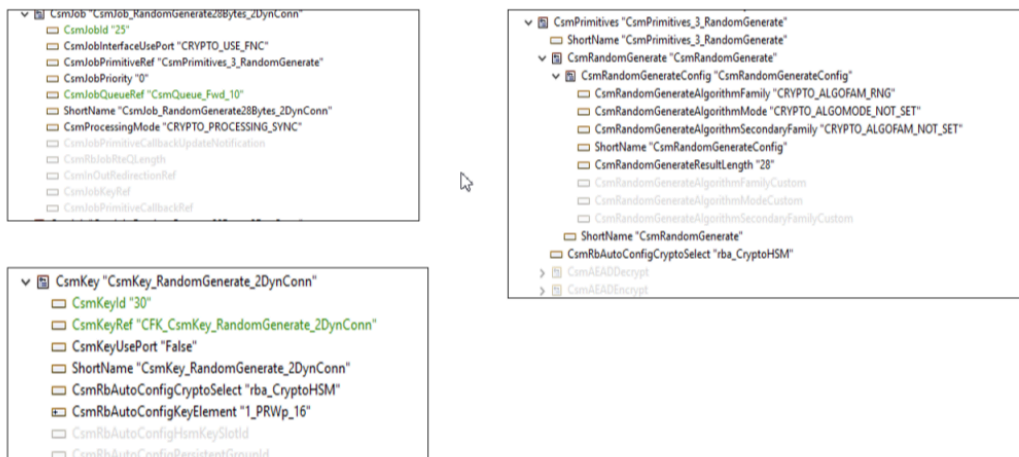


Figure 4.18.Required Configurations for Generation of Random Number from Csm

Configuration :Csm Signature Generate Configuration



Figure 4.19.Required Configurations for Generation of Signature

Configuration :Csm Hash Job 2 Configuration

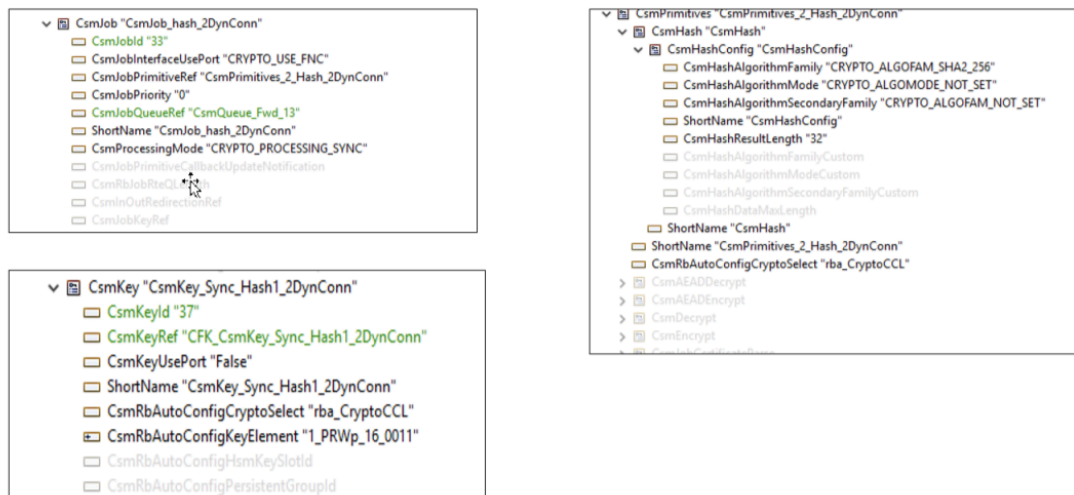


Figure 4.20.Required Configurations for Generation of Hash

4.6.3 RBA_ETHTLS_ECDH_RSA_WITH_AES_128_CBC_SHA256 Cipher-suite Configurations

The configurations, job and key references to be done in the *rba_EthTls* component to *Csm* and *KeyM* are specified as follows:

Configuration : Tls config container

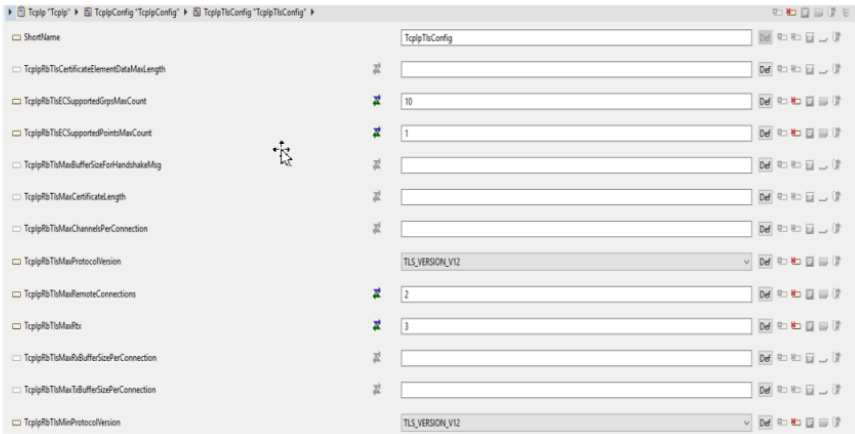


Figure 4.21.rba_EthTls Configurations to be done

Configuration : Tls config container

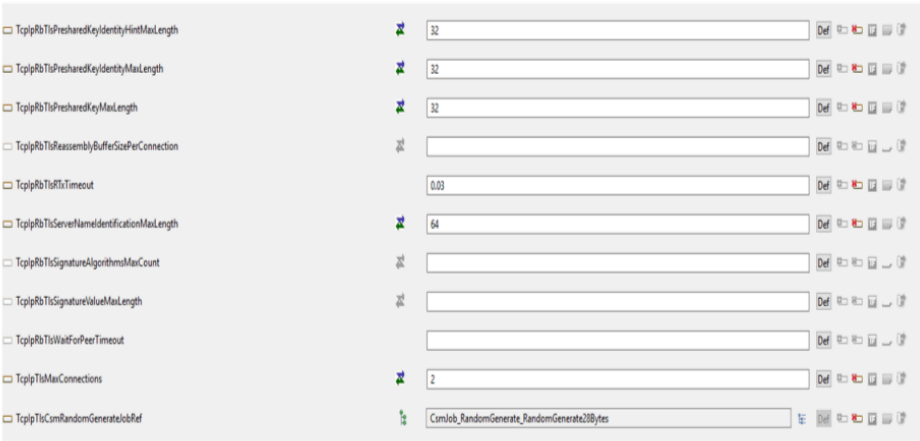


Figure 4.22.rba_EthTls Configurations to be done

Configuration : TcplpTlsCertificateIdentity container

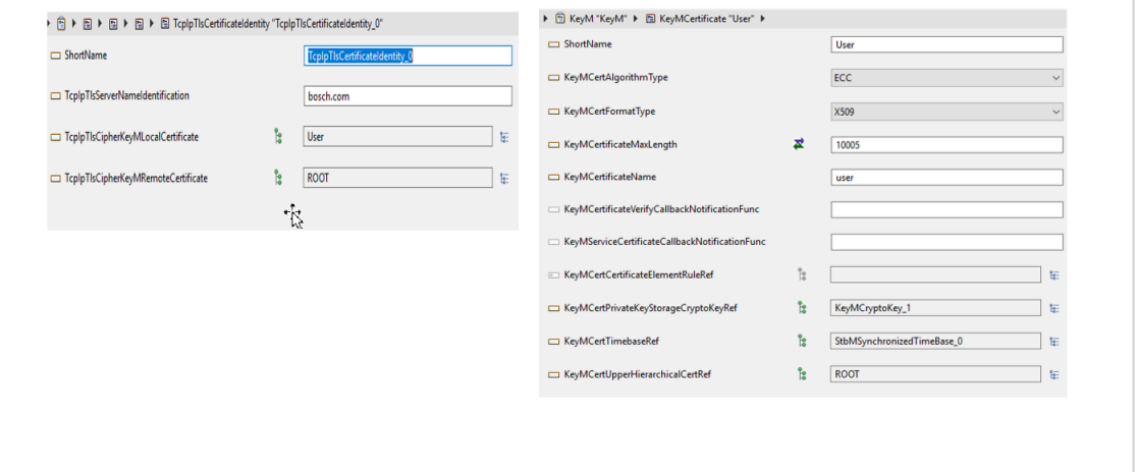


Figure 4.23.rba_EthTls Certificate Configurations to be done

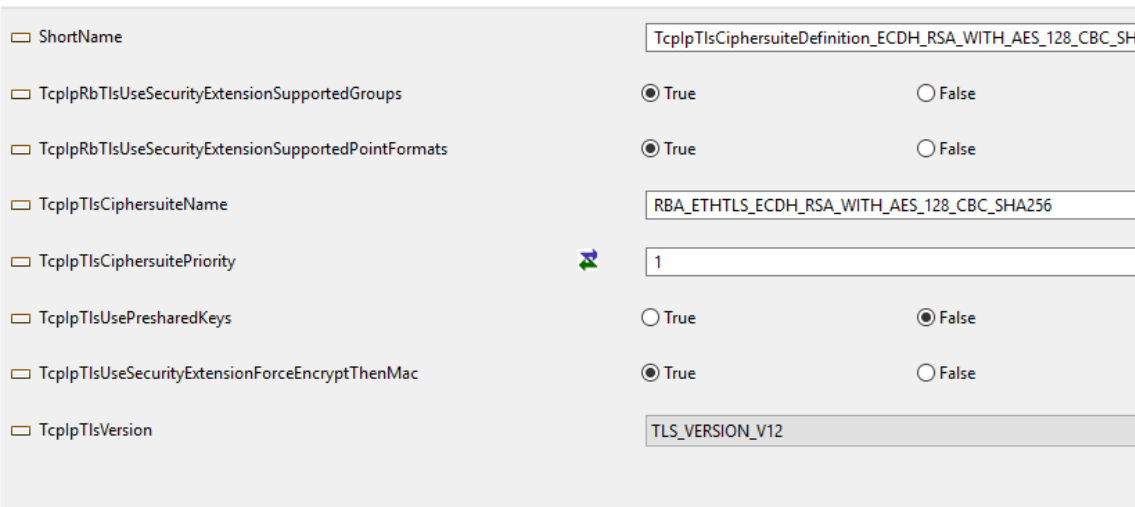


Figure 4.24.rba_EthTls Ciphersuite Definition Configurations to be done

ShortName	TcplpTlsCiphersuiteWorker_ECDH_RSA_WITH_AES_128_GCM_SHA256
TcplpTlsCipherCsmDecryptJobRef	CsmJob_Decrypt
TcplpTlsCipherCsmDecryptKeyRef	CsmKey_Decrypt
TcplpTlsCipherCsmEncryptJobRef	CsmJob_Encrypt
TcplpTlsCipherCsmEncryptKeyRef	CsmKey_Encrypt
TcplpTlsCipherCsmMacGenerateJobRef	CsmJob_MacGenerate_SHA256
TcplpTlsCipherCsmMacGenerateKeyRef	CsmKey_Sync_Mac_Generate_SHA256
TcplpTlsCipherCsmMacVerifyJobRef	CsmJob_MacVerify
TcplpTlsCipherCsmMacVerifyKeyRef	CsmKey_MacVerify
TcplpTlsCiphersuiteDefinitionRef	TcplpTlsCiphersuiteDefinition_ECDH_RSA_WITH_AES_128_CBC_SHA
TcplpTlsConnectionHandshakeRef	TcplpTlsHandshake_ECDH_RSA_WITH_AES_128_GCM_SHA256

Figure 4.25.rba_EthTls Ciphersuite Worker Configurations to be done

ShortName	TcplpTlsHandshake_ECDH_RSA_WITH_AES_128_GCM_SHA256	D
TcplpTlsCsmPRFSupportType	TLS_PRF_CSM_FULL_SUPPORT	D
TcplpTlsCsmHashVerifyJobRef	CsmJob_hash1	D
TcplpTlsCsmHashVerifyJobRef	CsmJob_hash	D
TcplpTlsCsmKeyExchangeCalcPubValJobRef		D
TcplpTlsCsmKeyExchangeCalcSecretJobRef	CsmJob_CalcPubVal_ED25519	D
TcplpTlsCsmKeyExchangeKeyRef	CsmKey_CalcPubVal_ED25519	D
TcplpTlsCsmKeyExchangeSignatureGenerateJobRef		D
TcplpTlsCsmKeyExchangeSignatureGenerateKeyRef		D
TcplpTlsCsmPrfMacJobRef	CsmJob_MacGenerate_HSM_HMAC_SHA256	D
TcplpTlsCsmPrfMacKeyRef	CsmKey_Sync_Mac_Hsm_HMAC_SHA256	D

Figure 4.26.rba_EthTls Handshake Container Configurations to be done

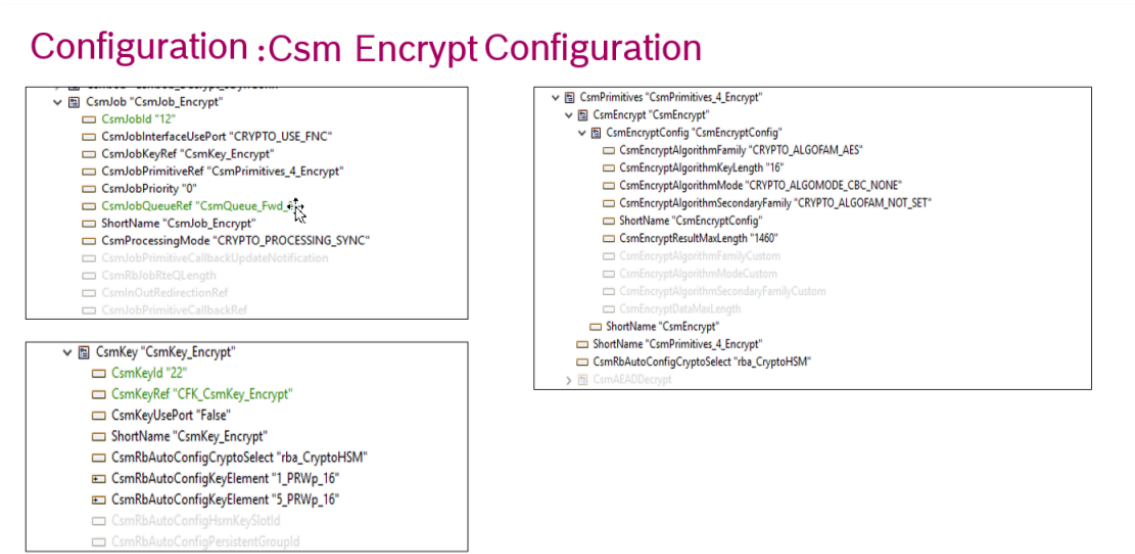


Figure 4.31.Required Configurations for Encryption Process

Error opening image file: [Errno 2] No such file or directory: '/c/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Mac_Generate_Configuration.png'

Figure 4.32.Required Configurations for Generation of MAC

Error opening image file: [Errno 2] No such file or directory: '/c/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Mac_Verify_Configuration.png'

Figure 4.33.Required Configurations for Verification of MAC

Configuration :Csm PRF Configuration

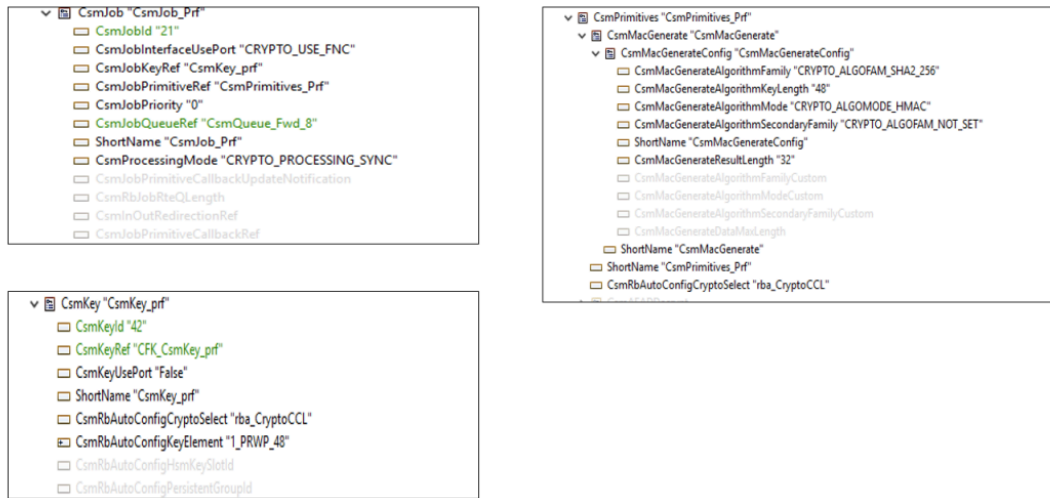


Figure 4.34.Required Configurations for PRF

Configuration :Csm Random Generate Configuration

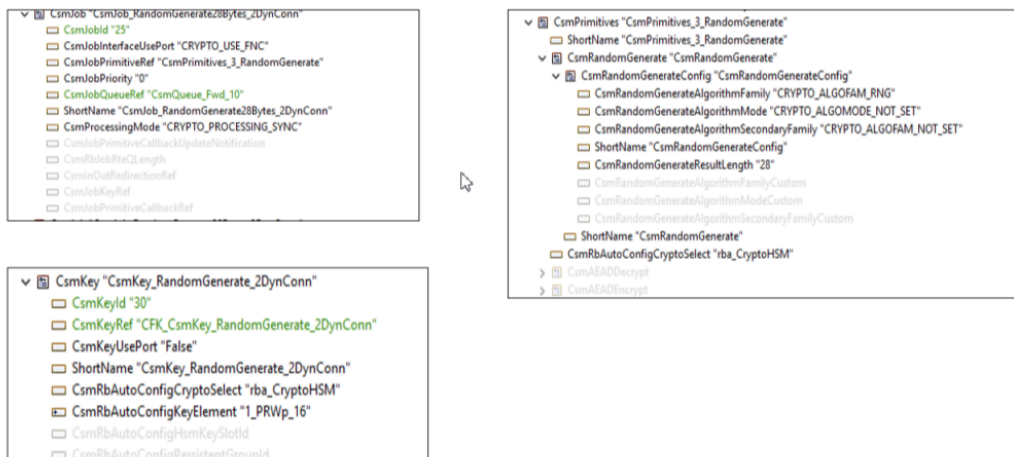


Figure 4.35.Required Configurations for Generation of Random Number from Csm

Configuration :Csm Signature Generate Configuration



Figure 4.36.Required Configurations for Generation of Signature

Configuration :Csm Hash Job 2 Configuration

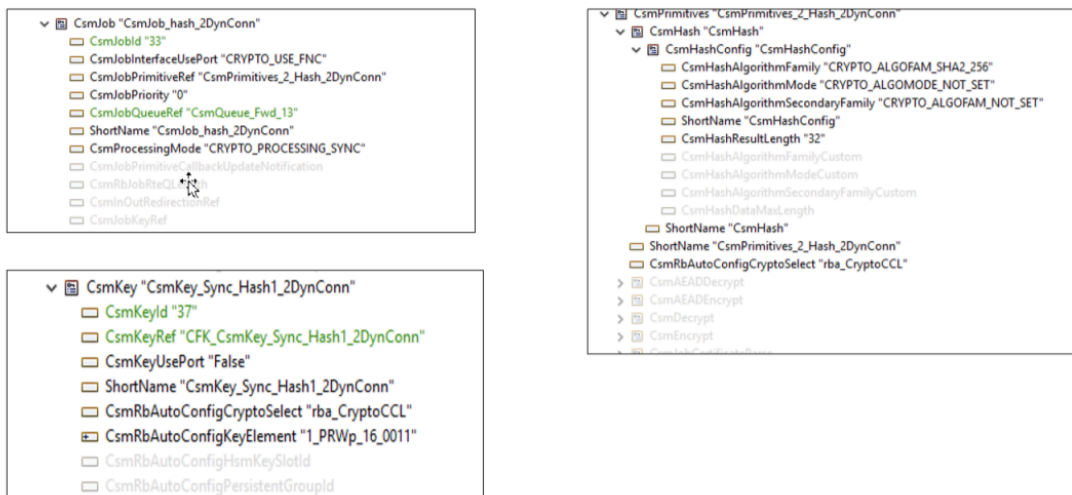


Figure 4.37.Required Configurations for Generation of Hash

4.6.4 RBA_ETHTLS_TLS13_AES_256_GCM_SHA_384 Ciphersuite Configurations

The configurations, job and key references to be done in the *rba_EthTls* component to *Csm* and *KeyM* are specified as follows:

Configuration: Tls Config Container

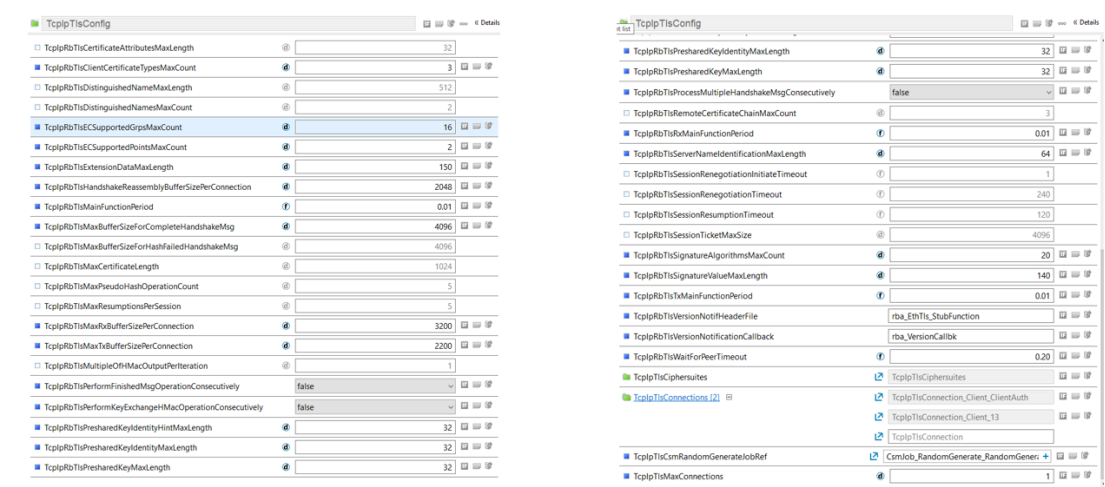


Figure 4.38.rba_EthTls Configurations to be done

Configuration: TcplpTlsCertificateIdentity Container

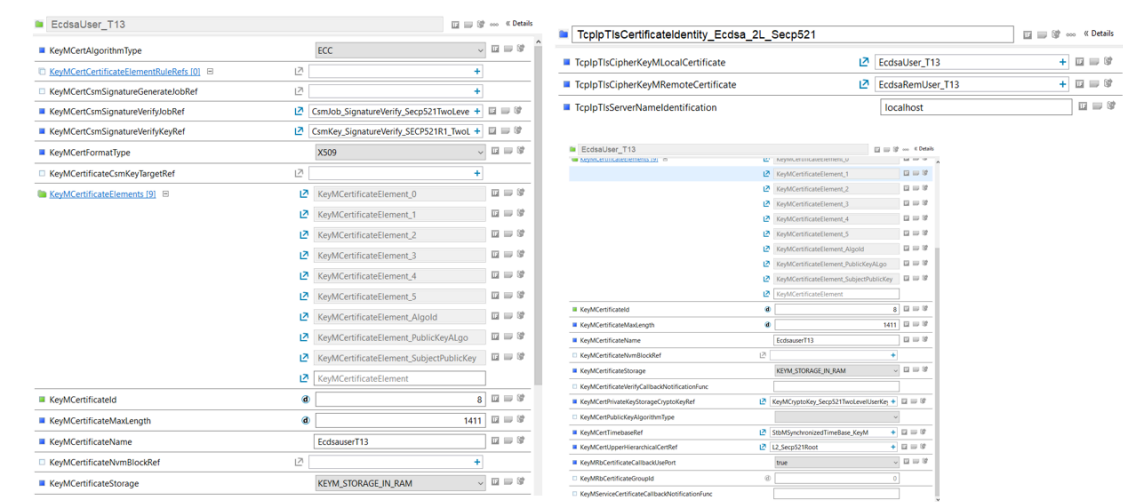


Figure 4.39.rba_EthTls Certificate Configurations to be done

Configuration: TcplpTlsCiphersuiteDefinition Container

TcplpTlsCiphersuiteDefinition_AES_256_GCM_SHA384

TcplpRbTlsUseSecurityExtensionSupportedGroups	true
TcplpRbTlsUseSecurityExtensionSupportedPointFormats	true
TcplpTlsCiphersuiteName	RBA_ETHTLS_AES_256_GCM_SHA384
TcplpTlsCiphersuitePriority	1
TcplpTlsUsePresharedKeys	false
TcplpTlsUseSecurityExtensionForceEncryptThenMac	false
TcplpTlsVersion	TLS_VERSION_V13

Figure 4.40.rba_EthTls Ciphersuite Worker Configurations to be done

Configuration: TcplpTlsCiphersuiteWorker Container

TcplpTlsCiphersuiteWorker_AES_GCM_256_SHA384

TcplpTlsCipherCsmDecryptJobRef	CsmJob_Decrypt_AES_GCM_256
TcplpTlsCipherCsmDecryptKeyRef	CsmKey_Decrypt_AES_GCM_256
TcplpTlsCipherCsmEncryptJobRef	CsmJob_Encrypt_AES_GCM_256
TcplpTlsCipherCsmEncryptKeyRef	CsmKey_Encrypt_AES_GCM_256
TcplpTlsCipherCsmMacGenerateJobRef	CsmJob_MacGenerate_SHA384
TcplpTlsCipherCsmMacGenerateKeyRef	CsmKey_MacGenerate_SHA384
TcplpTlsCipherCsmMacVerifyJobRef	CsmJob_MacVerify_SHA384_T13
TcplpTlsCipherCsmMacVerifyKeyRef	CsmKey_MacVerify_SHA384_T13
TcplpTlsCiphersuiteDefinitionRef	TcplpTlsCiphersuiteDefinition_AES_256_Gc
TcplpTlsConnectionHandshakeRefs [1]	TcplpTlsHandshake_Secp521r1_GCM_256_

Figure 4.41.rba_EthTls Ciphersuite Worker Configurations to be done

Configuration: TcplpTlsHandshake Container

TcplpTlsHandshake_Secp521r1_GCM_256_SHA384

TcplpRbTlsCsmKeyElementSetJobRef		+
TcplpRbTlsCsmKeyExchangeKeyPairKeyRef		+
TcplpTlsCsmHashVerifyJobRefs [1]	CsmJob_HashVerifyOp_SHA384	+ [icon] [icon]
		+
TcplpTlsCsmKeyExchangeCalcPubValJobRef	CsmJob_Secp521r1	+ [icon] [icon]
TcplpTlsCsmKeyExchangeCalcSecretJobRef	CsmJob_Secp521r1_SharedSec	+ [icon] [icon]
TcplpTlsCsmKeyExchangeKeyRef	CsmKey_Secp521r1	+ [icon] [icon]
TcplpTlsCsmKeyExchangeSignatureGenerateJobRef	CsmJob_SignatureGenerate_Secp521	+ [icon] [icon]
TcplpTlsCsmKeyExchangeSignatureGenerateKeyRef	CsmKey_SignGenerate_521r1	+ [icon] [icon]
TcplpTlsCsmKeyExchangeSignatureVerifyJobRef	CsmJob_SignatureVerify_Secp521	+ [icon] [icon]
TcplpTlsCsmKeyExchangeSignatureVerifyKeyRef	CsmKey_SignVerify_521r1	+ [icon] [icon]
TcplpTlsCsmPrfMacJobRef	CsmJob_Pr_SHA384	+ [icon] [icon]
TcplpTlsCsmPrfMacKeyRef	CsmKey_Pr_SHA384	+ [icon] [icon]
TcplpTlsCsmPRFSupportType		▼

Figure 4.42.rba_EthTls Handshake Container Configurations to be done

Configuration: Csm Public Value

CsmJob_Secp521r1

CsmInOutRedirectionRef		+
CsmJobId	41	[icon] [icon]
CsmJobInterfaceUsePort	CRYPTO_USE_FUNC	[icon] [icon]
CsmJobKeyRef	CsmKey_Secp521r1	+ [icon] [icon]
CsmJobPrimitiveCallbackRef	CsmCallback_0	+ [icon] [icon]
CsmJobPrimitiveRef	CsmPrimitives_Secp521r1_PubKey	+ [icon] [icon]
CsmJobPriority	0	[icon] [icon]
CsmJobQueueRef	CsmQueue_Fwd_36	+ [icon] [icon]
CsmProcessingMode	CRYPTO_PROCESSING_ASYNC	[icon] [icon]
CsmRbJobRteQLength	1	[icon] [icon]

CsmJobKeyExchangeCalcPubValConfig

CsmJobKeyExchangeCalcPubValAlgorithmFamily	CRYPTO_ALGOFAM_ECDSA	[icon] [icon]
CsmJobKeyExchangeCalcPubValAlgorithmFamilyCustomRef		+
CsmJobKeyExchangeCalcPubValAlgorithmMode	CRYPTO_ALGOMODE_NOT_SET	[icon] [icon]
CsmJobKeyExchangeCalcPubValAlgorithmModeCustomRef		+
CsmJobKeyExchangeCalcPubValAlgorithmSecondaryFamily	CRYPTO_ALGOFAM_CUSTOM	[icon] [icon]
CsmJobKeyExchangeCalcPubValAlgorithmSecondaryFamilyCustomRef	CRYPTO_ALGOFAM_EC_SECP521R1	+ [icon] [icon]

CsmKey_Secp521r1

CsmKeyId	39	[icon] [icon]
CsmKeyRef	CFK_CsmKey_Secp521r1	+ [icon] [icon]
CsmKeyUsePort	false	[icon] [icon]
CsmRbAutoConfigCryptoNvBlockRef		+
CsmRbAutoConfigCryptoSelect	rba_CryptoCCL	[icon] [icon]
CsmRbAutoConfigHsmKeySlotId		[icon] [icon]
CsmRbAutoConfigKeyElements [4]	10_PRWP_66	[icon] [icon]
	1_PRWP_66	[icon] [icon]
	12_PRWP_1_91	[icon] [icon]
	9_PRWP_66	[icon] [icon]
		[icon] [icon]

CsmCallback_0

CsmCallbackFunc	rba_EthTls_CsmAsyncCallback	[icon] [icon]
-----------------	-----------------------------	---------------

Figure 4.43.Required Configurations for Public Value Calculation

Configuration: Csm Public Secret

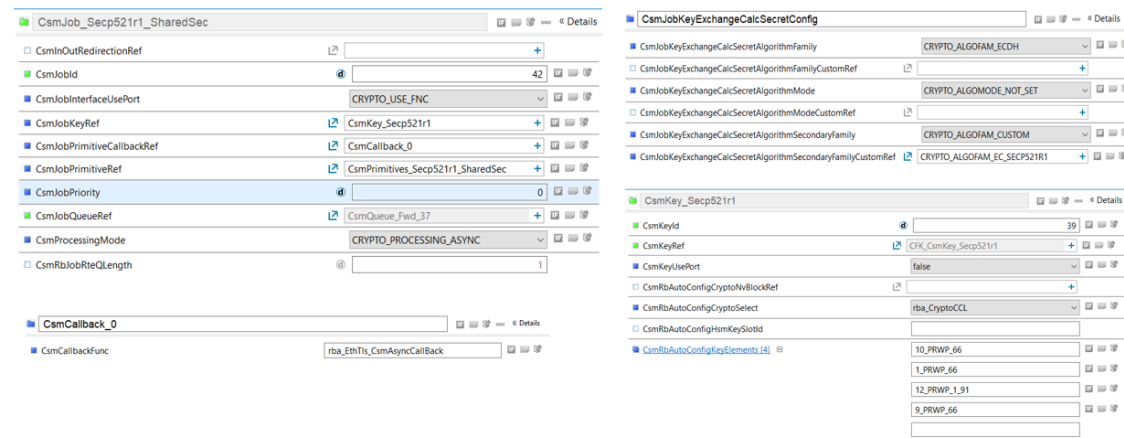


Figure 4.44.Required Configurations for Public Secret Calculation

Configuration: Csm Hash

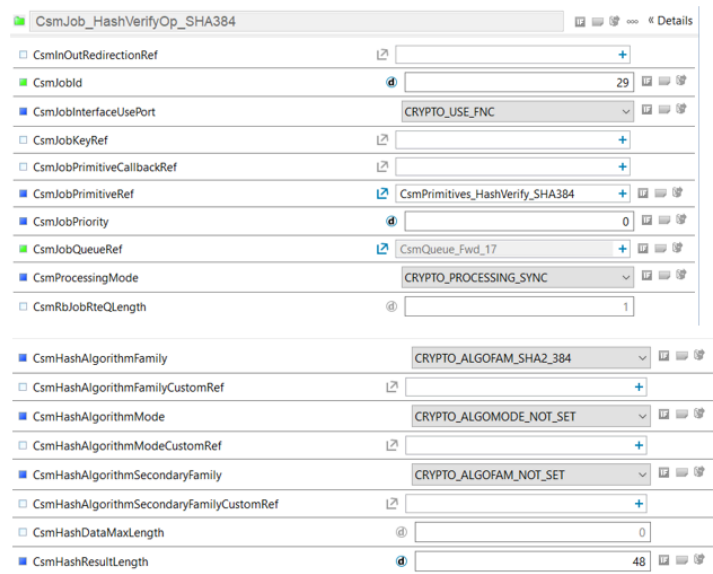


Figure 4.45.Required Configurations for Hash Calculation

Configuration: Csm Decrypt

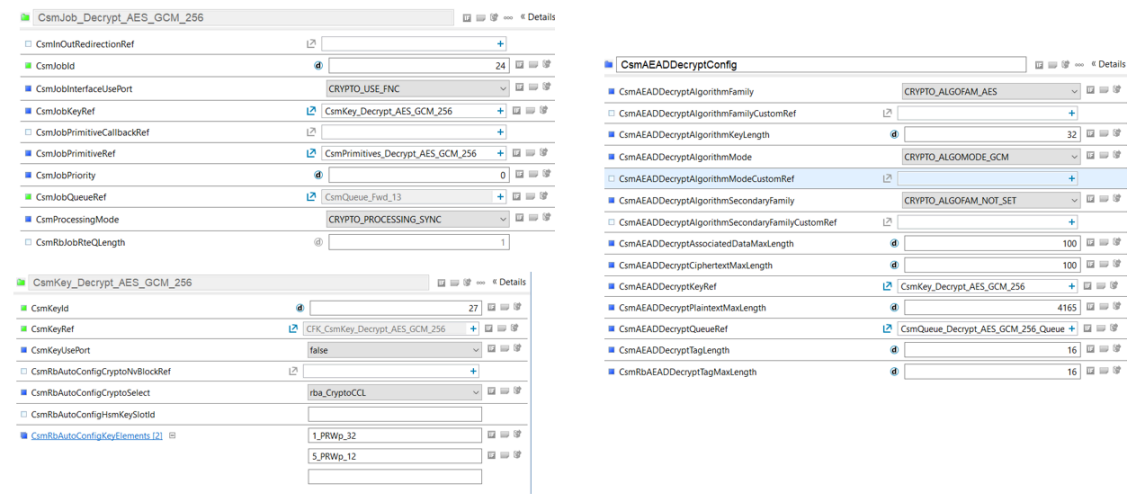


Figure 4.46.Required Configurations for Decryption Process

Configuration: Csm Encrypt

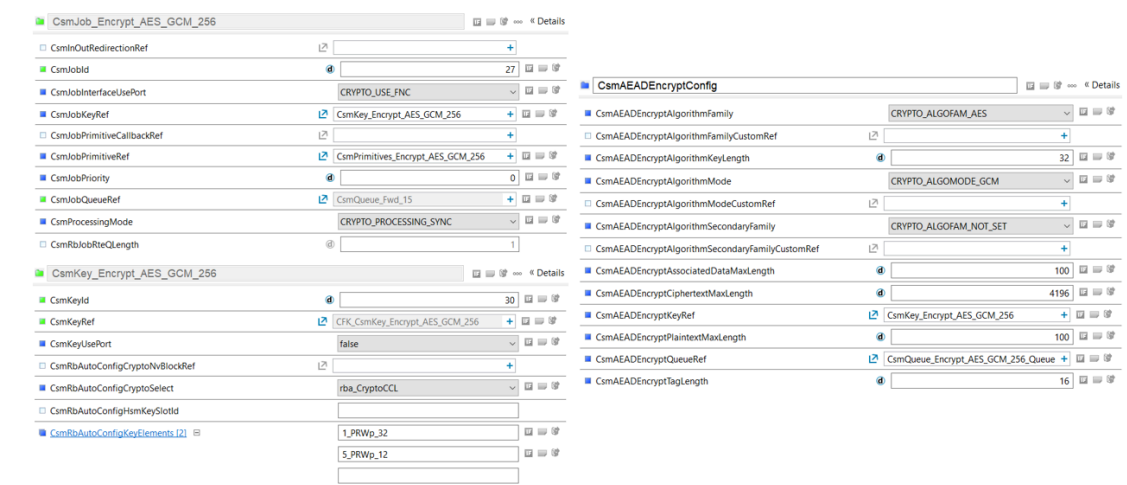


Figure 4.47.Required Configurations for Encryption Process

Error opening image file: [Errno 2] No such file or directory: 'c:/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Tls13_CsmMacGenerate.png'

Figure 4.48.Required Configurations for Generation of MAC

Error opening image file: [Errno 2] No such file or directory: 'c:/ws/Documents/Deliver-

Error opening image file: [Errno 2] No such file or directory: 'c/ws/Documents/Deliverables/Reference Guide/RTA-Components/RTA-Eth_Sec/source/modules/rba_EthTls/images/rba_EthTls_Tls13_CsmMacVerify.png'

Figure 4.49.Required Configurations for Verification of MAC

Configuration: Csm SignatureGenerate

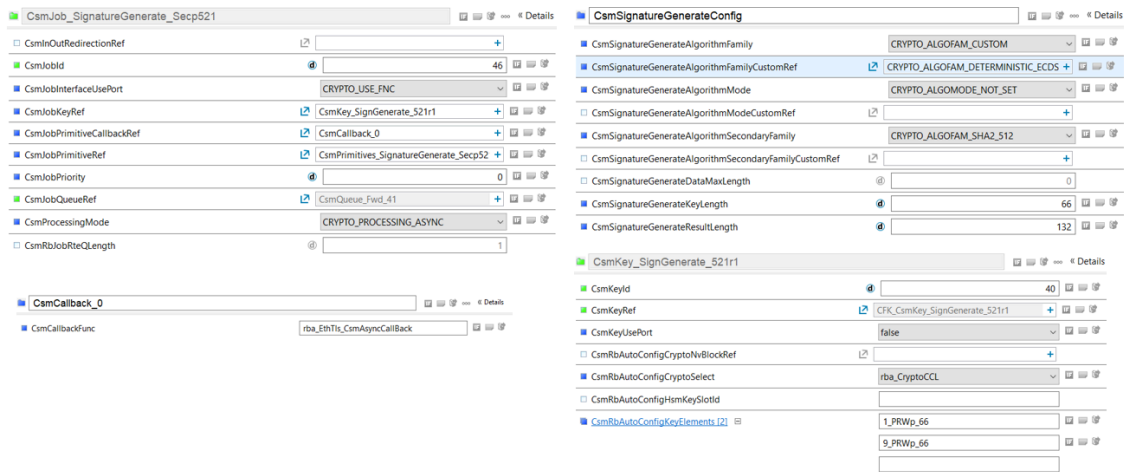


Figure 4.50.Required Configurations for Generation of Signature

Configuration: Csm SignatureVerify

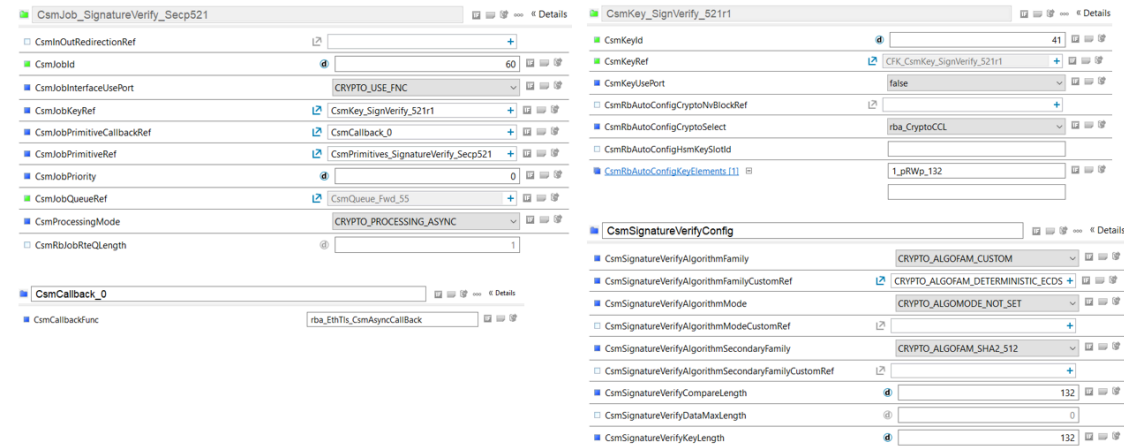


Figure 4.51.Required Configurations for Verification of Signature

4.6.5 General Configuration Information

- The configuration parameter TcpIpTlsCiphersuiteId is automatically forwarded in the TLS container based upon TcpIpTlsCiphersuiteNameConfigured.

- The parameter
`TcpIpRbTlsContinueHandshakeIfEmptyClientCertificateRcvd`,
`TcpIpRbTlsRequestCertificateAuthority` and
`TcpIpRbTlsUseRootCertInRequestCertificateAuthority` should be configured only if Client authentication for the TLS Server is configured.
- The parameters `TcpIpRbTlsClientCertificateTypesMaxCount`,
`TcpIpRbTlsDistinguishedNameMaxLength` and
`TcpIpRbTlsDistinguishedNamesMaxCount` should be configured only for the TLS Server with Client authentication enabled, or for the TLS Client with authentication mode enabled.
- The parameters `TcpIpRbTlsECSupportedGrpsMaxCount` and
`TcpIpRbTlsECSupportedPointsMaxCount` should be configured only if Elliptic curve ciphersuites are configured.
- The parameters `TcpIpRbTlsSessionRenegotiationCertComparisonEnabled`,
`TcpIpRbTlsSessionRenegotiationEnabled` and
`TcpIpRbTlsSessionRenegotiationSCSVCiphersuiteEnabled` should be configured only if session renegotiation is configured.
- The parameters `TcpIpRbTlsSessionResumptionTimeout`,
`TcpIpRbTlsMaxResumptionsPerSession`,
`TcpIpRbTlsDisableSessionResumptionOnSessionRenegotiation`,
`TcpIpRbTlsSessionResumptionMethod` and
`TcpIpRbTlsSessionTicketMaxSize` should be configured only if session resumption is configured.
- If a pre-Shared key-related ciphersuite is configured, then
`TcpIpTlsUsePresharedKeys`, the `TcpIpTlsPresharedKeyIdentity` container,
`TcpIpRbTlsPresharedKeyIdentityHintMaxLength`,
`TcpIpRbTlsPresharedKeyIdentityMaxLength` and
`TcpIpRbTlsPresharedKeyMaxLength` should be configured.
- The `TcpIpTlsCertificateIdentity` container and configuration parameters
`TcpIpRbTlsCertificateAttributesMaxLength`,
`TcpIpRbTlsMaxCertificateLength` and
`TcpIpRbTlsRemoteCertificateChainMaxCount` should be configured when Authentication is required for the ciphersuites.
- The configuration parameters
`TcpIpRbTlsUseSecurityExtensionRecordSizeLimitOfRemote` and
`TcpIpRbTlsUseSecurityExtensionMaxFragmentLengthOfRemote` should be configured if `RecordSizeLimit`` and `MaxFragmentLength` extensions have to be processed from Remote.
- The configuration parameter
`TcpIpRbTlsUseSecurityExtensionSignatureAlgorithm` should be configured if a Signature algorithms extension has to be sent to the Remote.
- The configuration parameter
`TcpIpRbTlsPerformKeyExchangeHMacOperationConsecutively` should be configured to split the HMAC operations in multiple main functions to reduce the runtime.

- The configuration parameter `TcpIpRbTlsPerformFinishedMsgOperationConsecutively` should be configured to prepare/process the finished message in multiple main functions to reduce the runtime.
- The configuration parameter `TcpIpRbTlsProcessMultipleHandshakeMsgConsecutively` should be configured to process multiple handshake messages consecutively in the same main function context to reduce the runtime of the TLS main function.
- The configuration parameter `TcpIpRbTlsMaxPseudoHashOperationCount` can be configured to specify the maximum number of *Pseudo Hash* operations that could be performed in the same main function.
- The configuration parameter `TcpIpTlsCsmHashVerifyJobRef`` should be configured with only one hash job (Async/Sync) when `TcpIpRbTlsMaxBufferSizeForCompleteHandshakeMsg` is greater than zero. Else, two hash jobs should be configured, with at least one job with CCL crypto libraries.

**NOTE**

If multiple hash jobs are not possible to enable parallel execution, all handshake messages should be buffered until the finished messages have been transmitted and received.

- The configuration parameter `TcpIpRbTlsMainFunctionPeriod` should be configured to forward the main function period to *rba_ArxmlGen*, which handles the secure connection establishment and closure (TLS handshake). In other words, this main function exchanges messages such as *Handshake*, *Change Cipher Spec* and *Alert*.
- The configuration parameter `TcpIpRbTlsTxMainFunctionPeriod` should be configured to forward the Tx main function period to *rba_ArxmlGen*, which handles the transmission functionality. This main function transmits the application data alone (i.e. the handshake messages won't be transmitted from here).
- The configuration parameter `TcpIpRbTlsRxMainFunctionPeriod` should be configured to forward the Rx main function period to *rba_ArxmlGen*, which handles the reception functionality. This main function processes the received application data alone (i.e. the handshake messages won't be processed from here).
- The configuration parameter `TcpIpRbTlsCsmKeyElementSet.JobRef` should be configured if the Edward curve is configured for the signature generate operation in any of the handshake.
- The configuration parameter `TcpIpRbTlsCsmKeyExchangeKeyPairKeyRef` should be configured for the TLS Server (for the ECDH key exchange cipher-suite).
- To avoid overwriting Csm jobs and keys when multiple connections are active, unique Csm jobs and keys should be configured in

TcpIpTlsCiphersuiteWorkersandTcpIpTlsSessiontickets.

Table 4.3.General Configuration Information

Configuration parameters	rba_CryptoHSM	rba_CryptoCCL
Csm_RandomGenerate	Yes	No
Csm_MacGenerate	Yes	Yes
Csm_MacVerify	Yes	Yes
Csm_MacGenerate (Key Derivation)	No	Yes
Csm_Hash	Yes	Yes
Csm_Encrypt	Yes	Yes
Csm_Decrypt	Yes	Yes
Csm_KeyExchangeCalcPubVal	Yes	No
Csm_SignatureGenerate	Yes	No
Csm_SignatureVerify	Yes	No
Csm_KeyExchangeCalcSecret	Yes	No

- The ciphersuite configuration preference in TLS should be in the order of strong ciphersuites to weak ciphersuites.
- The same maximum fragment length should be configured in both of the nodes.
- Configuring the TcpIpTlsCiphersuiteName parameter and running the TLS validation scripts will redirect you via validation errors to configure the other parameters. (This might be helpful for an integrator who is new to TLS.)
- Multiple Clients connecting to same Server results in delayed handshake completion for the second Client, because the same remote certificate identity is used for the Client certificate verification. To improve performance, more than one remote certificate identity can be configured in the Server connection to perform the Client certificate verification in parallel (via a different identity).
- If the Background MainFunction is enabled, then its entry should be added into the RTE ECU values, otherwise it should be removed.
- If TcpIpRbTlsProcessMultipleHandshakeMsgConsecutively is set to TRUE when either the *MacVerify* or *Decrypt Csm job* are configured synchronously, then a double task activation issue may occur if continuous packets are received. Instead, set TcpIpRbTlsProcessMultipleHandshakeMsgConsecutively to FALSE if either *MacVerify* or *Decrypt Csm job* are configured synchronously.

4.6.6 TLS 1.3 Specific Configuration Advice

- Currently HKDF key derivation does not support separate execution of HKDF extract and expand operations, therefore no separate jobs are to be configured for HKDF operation. An implementation of HKDF has been done inter-

nally using *ecy_lib* (available through *rba_EthTls_Tls13_Crypto.c_tpl* and *rba_EthTls_Tls13_Crypto.h_tpl*) to support key generation. This template can be used as a reference and when included in the project will enable a successful TLS 1.3 handshake.

- Mac Generate and Hash jobs must be configured only in Sync mode in TLS 1.3 Handshake
- The parameter *TcpIpRbTlsMaxBufferSizeForCompleteHandshakeMsg* should be configured with a value greater than 0. This is required as the current hash service does not support intermediate hash generation during STREAM mode operation.
- Currently only the following crypto services can be configured in cipher-suite:

AES_256_GCM_SHA_384Key exchange

ECDHE SECP521R1Signature

ECDSA SECP521R1Mac operation

HMAC SHA384Key derivation

HKDF SHA384

4.6.7 Det Events

rba_EthTls reports detected development errors to the *Det_ReportError* service of the Development Error Tracer (DET) module. The detection of development errors is configured (ON / OFF) at pre-compile time by setting the switch *TCPIP_DEV_ERROR_DETECT*.



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

4.7 Integration Advice

4.7.1 How To Integrate

Integration and configuration of *rba_EthTls* is part of the same process for *Tcplp* i.e. there is no separate configurable module. Instead, a project software/system description including *Tcplp* is created and an ECUC generated - with the User then selecting *rba_EthTls* for the RTA-CAR Code Generation.

4.7.2 Init Functions

rba_EthTls_Init is called by *EcuM*. It is also used to re-initialize the module.

4.7.3 Delnit Functions

A separate shutdown API is not supported.

4.7.4 Scheduled Functions

TcpIp_MainFunction is to be scheduled. rba_EthTls_MainFunction is called from TcpIp_MainFunction.

4.7.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthTls module uses the following OS resources:

- rba_EthTls_AtomicOperation
- rba_EthTls_ReserveDynConn
- rba_EthTls_ReservePseudoHashOpIdx
- rba_EthTls_ReserveSessionResumptionTbl
- rba_EthTls_ConnStateHandlers
- rba_EthTls_CsmJobToTlsDynConnMap
- rba_EthTls_KeyMCertIdToTlsDynConnMap
- rba_EthTls_TransmittedLength
- rba_EthTls_TxBuffer
- rba_EthTls_RxBuffer
- rba_EthTls_TimerHandling
- rba_EthTls_ReserveCiphersuiteWorker
- rba_EthTls_ReserveRemoteCertificateId

4.8 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

5 Glossary

Activation Line

A routing path for diagnostic messages

API Application Programming Interface

ARP

Address Resolution Protocol. A communication protocol used for discovering link layer (MAC) addresses

BswM

AUTOSAR BSW Mode Manager, supports those aspects of Vehicle Mode Management and Application Mode Management that are applicable for BSW

CBV

Control Bit Vector. An array of Boolean values which signal the control properties of an NM-PDU

CDD

Complex Device Driver. A software entity not standardised by AUTOSAR, but which can access (or be accessed by) AUTOSAR interfaces and/or RTA-BSW module APIs

Channel

A logical bus connection to a particular node or ECU

Cluster

A communication system made up of a number of nodes linked by a physical transmission medium

ComM

AUTOSAR Communication Manager, provides the control of the underlying communication services

CRC

Cyclic Redundancy Checksum

DHCP

Dynamic Host Configuration Protocol. A network management protocol used in UDP/IP networks

DoIP

AUTOSAR Diagnostic Over IP module

EthIf

AUTOSAR Ethernet Interface module

EthSM

AUTOSAR Ethernet State Manager module

EthTSyn

AUTOSAR Time Sync Over Ethernet module

Gratuitous ARP

An unrequested ARP reply, normally used to announce the presence of a new network device

ICMP

Internet Control Message Protocol. A supporting protocol used in the Internet protocol suite to send error messages and operational information

IP Internet Protocol. The principal communications protocol of the Internet protocol suite

IP Address

A numerical label assigned to each device on an IP network. In IPv4, the IP address is a 32-bit value

IPv4

First major version of the Internet Protocol

IPv6

Latest major version of the Internet Protocol

I-PDU

PDU of the interaction layer

Link-local

A type of connection which links devices located on the same physical or logical network

MAC Address

Media Access Control address. A unique ID number which identifies a device on a network and is usually assigned at the hardware level

MSS

Maximum Segment Size. The largest amount of data that can be received in a single TCP segment, specified in octets

MTU

Maximum Transmission Unit

NDP

Neighbor Discovery Protocol

Nm AUTOSAR Network Management Interface, responsible for coordinated ECU network access, in particular, the transition to sleep mode

NM-PDU

Network Management PDU. A PDU containing the payload of a network management message, consisting of the user data, CBV and source node identifier

Node

An ECU which is connected to a communications network

N-SDU

PDU of the network transport layer

OFS

Offset Synchronization

PDU

Protocol Data Unit

PduR

AUTOSAR PDU Router, responsible for routing PDUs between communication modules and/or transport protocol modules

Port

A physical or logical endpoint on a computer network

PTP

Precision Time Protocol

rba_EthArp

RTA-BSW module, provides address resolution services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthAutoIp

RTA-BSW module, provides auto-configuration of local IP addresses which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthDHCP

RTA-BSW module, provides DHCP host configuration services which are used by the Tcplp module to assign host-specific configuration parameters and network addresses to host network devices

rba_EthICMP

RTA-BSW module, provides ICMP services which are used by the Tcplp module to provide error message and query communications between the SoAd and EthIf modules

rba_EthICMPv6

RTA-BSW implementation of AUTOSAR ICMPv6

rba_EthIPv4

RTA-BSW module, provides IPv4 protocol services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthIPv6

RTA-BSW implementation of AUTOSAR IPv6

rba_EthNdp

RTA-BSW implementation of AUTOSAR IPv6 NDP

rba_EthTcp

RTA-BSW module, provides TCP communication services used by the Tcplp module (acting as a wrapper) to provide intermediate level communication between the SoAd and EthIf modules

rba_EthUdp

RTA-BSW module, provides UDP protocol services are used by the Tcplp module to provide UDP data communication between the SoAd and EthIf modules

rba_SomelpXfAdp

RTA-BSW module, responsible for allocating the memory needed for data transformation and calling the transformer functions

RTO

Retransmission Timeout

Sd AUTOSAR Service Discovery module

SDU

Service Data Unit

SNMA

Solicited-Node Multicast Address, used in Neighbor Discovery Protocol (NDP) for obtaining link-layer addresses

SoAd

AUTOSAR Socket Adapter module

Socket

An internal endpoint for sending or receiving data within a node on a communications network

SOME/IP

Scalable Service-Oriented Middleware Over IP. An automotive middleware solution which can be used to send and receive control messages on a vehicle network

SomelpTp

AUTOSAR Scalable service-Oriented MiddlewarE over IP (SOME/IP) Transport Protocol module

SomelpXf

AUTOSAR SOME/IP Transformer module

SOME/IP-SD

The service discovery component of SOME/IP

StbM

AUTOSAR Synchronized Time-Base Manager module

Switch

A networking device which routes information via data packet switching

TCP

Transmission Control Protocol. One of the main communication protocols of the Internet protocol suite

TcpIp

AUTOSAR TCP/IP module

TLV

Time, length and value coding

TP-PDU

A PDU transmitted or received by a lower layer transport protocol module

Transformer

A transformer employs a data transformation mechanism to linearize and transform data

UDP

User Datagram Protocol. One of the main communication protocols of the Internet protocol suite

UdpNm

AUTOSAR UDP Network Management module

VIN Vehicle Identification Number

VLAN

Virtual Local Area Network